

RTL Design Sherpa

RLB Top Micro-Architecture Specification 1.0

September 30, 2026

Table of Contents

1 Rlb Top Mas Index....15
2 RLB Top - Overview....16
2.1 What This Block Is....16
2.2 Why It Is a Separate Specification....16
2.3 Block Inventory....16
2.4 Features....17
2.5 Deliberate Scope Boundaries....18
2.6 Applications....18
2.7 Related Documents....19
3 RLB Top - Architecture....20
3.1 Hierarchy....20
3.2 The Address Path....20
3.3 The Interrupt Fabric....21
3.3.1 Source to line....21
3.3.2 The three destinations....22
3.3.3 OR-ed in, never substituted....23
3.4 The Cascade Cross-Connect....23
3.5 The Aggregated Output....23
3.6 Boot-Interrupt Rerouting....23
3.7 Related Documents....24
4 RLB Top - Clocks and Reset....25
4.1 The Short Version....25

4.2 Clock and Reset Ports....25
4.3 Why the Extra Clocks Are Unused....25
4.4 Reset Behaviour....26
4.4.1 Reset state of the interrupt fabric....26
4.4.2 Resetting to clear an interrupt....27
4.5 Related Documents....27
5 RLB Top - Acronyms and Terminology....28
5.1 Acronyms....28
5.2 Terminology....29
5.3 Related Documents....30
6 RLB Top - References....31
6.1 External Specifications....31
6.2 In-Repository References....32
6.2.1 This subsystem....32
6.2.2 Per-block specifications....32
6.2.3 Repository practice....32
6.3 Related Documents....33
7 RLB Top - Block Hierarchy Overview....34
7.1 The Twelve Instances....34
7.2 u_apbx_xbar - the address decode....35
7.3 The two 8259 instances....35
7.4 u_ioapic - and what it is not wired to....36
7.5 u_ioapic_boot_intx - the boot-interrupt companion....36
7.6 The glue....36

7.7 Related Documents..... 37
8 RLB Top - The APB Crossbar Block..... 38
8.1 Overview..... 38
8.2 Generation..... 38
8.3 Slave Index to Window..... 38
8.4 Address Forwarding..... 39
8.5 Decode Miss Behaviour..... 39
8.6 What This Block Does Not Do..... 39
8.7 Related Documents..... 40
9 RLB Top - The Interrupt Fabric Block..... 41
9.1 Overview..... 41
9.2 The Routing Table..... 41
9.2.1 Convention versus choice..... 42
9.2.2 The HPET legacy pair..... 42
9.3 Distribution..... 42
9.4 What the Fabric Does Not Do..... 43
9.5 Verification Heritage..... 43
9.6 Related Documents..... 43
10 RLB Top - The 8259 Cascade Cross-Connect..... 45
10.1 Overview..... 45
10.2 Why the Pair Exists At All..... 45
10.3 The Three-Wire Cross-Connect..... 45
10.4 The Master: IR2 Is Forced, Not OR-ed..... 45
10.5 The Slave: An Inline Input Worth Naming..... 46

10.6 The Boot-Interrupt Side Effect....46
10.7 Verification Heritage....46
10.8 Related Documents....46
11 RLB Top - Top-Level Interface....48
11.1 Overview....48
11.2 Parameters....48
11.2.1 Why only two interrupt lines are parameters....49
11.3 Ports....50
11.3.1 Clock and Reset....50
11.3.2 APB4 Slave Interface....51
11.3.3 Peripheral External Interfaces....51
11.3.4 Ports that need care....53
11.4 Address Decode Contract....54
11.5 Integration Checklist....54
11.6 Related Documents....54
12 RLB Top - APB4 Protocol at the Subsystem Boundary....56
12.1 Overview....56
12.2 Protocol Feature Support....56
12.3 Transfer Phases....56
12.4 Wait States....57
12.5 Write Strobes and Protection....57
12.6 Error Responses....57
12.7 Rules an Integrator Must Honour....58
12.8 Related Documents....58

13 RLB Top - Interrupt Pin Reference....59
13.1 Overview....59
13.2 Block Interrupt Outputs, Pin by Pin....59
13.3 The Sixteen Legacy Lines As the Controllers See Them....62
13.4 External Interrupt Input Pins....63
13.5 The Aggregate....64
13.6 Scope Boundaries Restated....64
13.7 Related Documents....64
14 RLB Top - Initialization and Programming Guide....66
14.1 Scope....66
14.2 Address Helper....66
14.3 Bring-Up Order....67
14.4 Step 1: Confirm the Subsystem Answers....67
14.5 Step 2: Choose the Interrupt Topology....68
14.6 Step 3a: Single-Controller Configuration....68
14.7 Step 3b: Cascaded-Pair Configuration....69
14.8 Step 4: IOAPIC Redirection Entries....70
14.9 Step 5: Enable the Blocks....71
14.10 Step 6: Verify....71
14.11 Clearing an Interrupt....71
14.12 Related Documents....72
15 RLB Top - Use Cases....73
15.1 Use Case 1: A UART Interrupt Through the Master Controller....73
15.2 Use Case 2: A GPIO Interrupt Through the Cascade....73

15.3 Use Case 3: The Same Source Through the IOAPIC....74
15.4 Use Case 4: HPET Legacy Replacement....74
15.5 Use Case 5: Boot-Interrupt Rerouting....75
15.6 Use Case 6: One Interrupt Line for the Whole Subsystem....75
15.7 Coincident Sources....75
15.8 Related Documents....76
16 RLB Top - Power Management: Sleep and Wake Across Blocks....77
16.1 Overview....77
16.2 Who Participates....77
16.3 Step 1: Bring the PM Block Up....78
16.4 Step 2: Arm the Wake Sources....78
16.5 Step 3: Enter Sleep....79
16.6 Step 4: Wake....80
16.7 Step 5: The Interrupt Path for ACPI Events....80
16.8 What Sleep Does Not Disturb....81
16.9 Related Documents....81
17 RLB Top - Address Map....82
17.1 rlb_top Has No Registers....82
17.2 Decode....82
17.3 Window Map....82
17.4 Read-Safe Probe Registers....83
17.5 The IOAPIC Is Indirect....84
17.6 Interrupt Line Assignments....84
17.7 Related Documents....85

18 Retro Legacy Blocks - Product Requirements Document....86
18.1 1. Overview....86
18.1.1 1.1 Purpose....86
18.1.2 1.2 Design Philosophy....86
18.1.3 1.3 Target Applications....86
18.2 2. Implemented Blocks....87
18.2.1 2.1 HPET - High Precision Event Timer....87
18.3 3. Planned Blocks....88
18.3.1 3.1 8259 - Programmable Interrupt Controller (PIC)....88
18.3.2 3.2 8254 - Programmable Interval Timer (PIT)....88
18.3.3 3.3 GPIO - General Purpose I/O....88
18.3.4 3.4 RTC - Real-Time Clock....89
18.3.5 3.5 SMBus Controller....89
18.3.6 3.6 UART - Universal Asynchronous Receiver/Transmitter....89
18.3.7 3.7 SPI Controller....89
18.3.8 3.8 I2C Controller....90
18.3.9 3.9 Watchdog Timer....90
18.3.10 3.10 Power Management / ACPI Controller....90
18.3.11 3.11 IOAPIC - I/O Advanced Programmable Interrupt Controller....90
18.3.12 3.12 Interconnect ID / Version Registers....90
18.4 4. Integration and Wrapper Goals....91
18.4.1 4.1 Individual Block Integration....91
18.4.2 4.2 RLB Wrapper Architecture....92
18.5 5. Design Standards....94

18.5.1 5.1 Reset Handling....94
18.5.2 5.2 Register Generation....94
18.5.3 5.3 Testbench Architecture....94
18.5.4 5.4 FPGA Synthesis Attributes....95
18.5.5 5.5 Documentation Requirements....95
18.6 6. Quality Metrics....95
18.6.1 6.1 Production Readiness Criteria....95
18.6.2 6.2 Current Status....96
18.7 7. Development Roadmap....97
18.7.1 7.1 Phase 1: Foundation (Complete)....97
18.7.2 7.2 Phase 2: Core Peripherals (Next 6-9 Months)....97
18.7.3 7.3 Phase 3: Advanced Peripherals (9-15 Months)....97
18.7.4 7.4 Phase 4: System Integration (15+ Months)....98
18.8 8. References....98
18.8.1 8.1 External Standards....98
18.8.2 8.2 Internal Documentation....98
18.8.3 8.3 Block-Specific Documentation....98
18.9 9. Success Criteria....98
18.9.1 9.1 Individual Block Success....98
18.9.2 9.2 Collection Success....99
18.9.3 9.3 Long-Term Vision....99
19 RLB (Retro Legacy Blocks) - FPGA Implementation Guide....100
19.1 What Are RLB (Retro Legacy Blocks)?....100
19.1.1 Overview....100

19.1.2 Purpose....100
19.2 Implemented RLB Modules....100
19.2.1 Complete Modules....100
19.3 FPGA Implementation - Nexys A7 / Genesys2....102
19.3.1 Can RLB Be Implemented on These Boards? YES!....102
19.3.2 Target FPGA Specifications....102
19.3.3 Resource Usage Estimates....102
19.3.4 What Can Fit on These Boards?....103
19.4 RLB System Integration on FPGA....103
19.4.1 Typical FPGA System Architecture....103
19.4.2 Example Use Cases on FPGAs....104
19.5 Nexys A7 Specific Integration....105
19.5.1 Available Resources on Nexys A7....105
19.5.2 How RLB Modules Map to Nexys A7....105
19.5.3 Example Nexys A7 Top-Level....105
19.6 Implementation Recommendations....106
19.6.1 For Nexys A7-100T....106
19.6.2 For Genesys 2....106
19.7 FPGA-Specific Considerations....107
19.7.1 Clock Management....107
19.7.2 Power Management....107
19.7.3 Interrupt Routing....107
19.7.4 External Interfaces....107
19.8 Soft Processor Options....108

19.8.1 MicroBlaze (Xilinx)....108
19.8.2 RISC-V (Open Source)....108
19.8.3 Custom Soft Core....108
19.9 Step-by-Step FPGA Implementation....108
19.9.1 Phase 1: Basic System (Week 1-2)....108
19.9.2 Phase 2: Expand Peripherals (Week 3-4)....109
19.9.3 Phase 3: Advanced Features (Week 5-6)....109
19.9.4 Phase 4: Application Development (Week 7+)....109
19.10 Example Address Map for FPGA....109
19.11 Software Development....110
19.11.1 Bare-Metal Firmware....110
19.11.2 Operating System Support....110
19.12 Advantages of RLB on FPGA....110
19.12.1 Development Benefits....110
19.12.2 Technical Benefits....111
19.12.3 Vs. Microcontroller....111
19.13 Conclusion....111
19.13.1 Can RLB Be Used on Nexys A7/Genesys2? ABSOLUTELY!....111

List of Figures

No figures in this document.

List of Tables

No tables in this document.

List of Waveforms

No waveforms in this document.

1 Rlb Top Mas Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 RLB Top - Overview

2.1 What This Block Is

`rlb_top` is the integration level of the Retro Legacy Blocks subsystem. It takes one APB4 slave port and presents, behind it, a complete set of PC-compatible legacy peripherals: two interrupt controllers, an IOAPIC, two timers, a real-time clock, an SMBus controller, a GPIO controller, a UART, and a power-management block.

It is not a peripheral. It contains no registers of its own, implements no protocol, and every address it answers belongs to something it instantiates. What it does contribute is three things that exist nowhere else in the subsystem:

1. **An address map.** A 1-to-10 APB crossbar splits one slave port into ten 4KB windows from `BASE_ADDR`, and answers an unmapped address with `PSLVERR` rather than hanging.
2. **An interrupt fabric.** Each block's interrupt output is routed internally onto its conventional legacy line and delivered to both 8259s and to the IOAPIC, so a board does not have to wire ten interrupt pins back in.
3. **A cascade.** The two 8259 instances are cross-connected as a PC/AT master and slave pair, with the vector and acknowledge path between them.

2.2 Why It Is a Separate Specification

Each of the nine other books in this directory specifies one block. None of them can specify the above, because none of them can see it: a block knows its own interrupt output pin, not which controller that pin reaches, and certainly not what happens when two blocks assert at once.

Before this book existed, the routing table and the bring-up order lived only in the testbench under `dv/tbclasses/rlb_top/`. That is a workable place to verify behaviour and a poor place to look it up.

2.3 Block Inventory

Ten windows, all assigned. There is no reserved window.

Window	Address range	Block	Instance	Specification
0	`0xFEC0_0000` - `0xFEC0_0FFF`	HPET	`u_hpet`	hpet_mas
1	`0xFEC0_1000` - `0xFEC0_1FFF`	8259 PIC (master)	`u_pic`	pic_8259_mas
2	`0xFEC0_2000` -	8254 PIT	`u_pit`	pit_8254_mas

Window	Address range	Block	Instance	Specification
	0xFEC0_2FFF			
3	0xFEC0_3000 - 0xFEC0_3FFF	RTC	u_rtc	rtc_mas
4	0xFEC0_4000 - 0xFEC0_4FFF	SMBus	u_smbus	smbus_mas
5	0xFEC0_5000 - 0xFEC0_5FFF	PM/ACPI	u_pm_acpi	pm_acpi_mas
6	0xFEC0_6000 - 0xFEC0_6FFF	IOAPIC	u_ioapic	ioapic_mas
7	0xFEC0_7000 - 0xFEC0_7FFF	GPIO	u_gpio	gpio_mas
8	0xFEC0_8000 - 0xFEC0_8FFF	UART 16550	u_uart	uart_16550_mas
9	0xFEC0_9000 - 0xFEC0_9FFF	8259 PIC (cascade slave)	u_pic_slave	pic_8259_mas

Two further instances carry no address window:

Instance	Module	Contribution
u_apbx_xbar	apbx_xbar_1to10	The address decode. Generated, not hand-written.
u_ioapic_boot_intx	ioapic_boot_intx	Turns “the IOAPIC is not delivering this pin” into a legacy PIC input.

Twelve instances in total.

2.4 Features

- Single APB4 slave entry point, 32-bit address and data, with PSTRB and PPROT carried through to every block
- Ten 4KB windows on a clean power-of-two decode: the slave index is PADDR[15:12]
- Unmapped addresses complete with PSLVERR instead of stalling the bus
- Interrupt fabric covering six sourcing blocks, presented on the conventional legacy IRQ lines

- Both 8259s cross-connected as a cascaded pair, with master IR2 protected
- A single aggregated `rlb_irq_out` for a system that wants one interrupt input rather than ten
- Boot-interrupt rerouting available and disabled by default
- External `pic_irq_in` and `ioapic_irq_in` retained as inputs and OR-ed with the internal fabric, so an existing board keeps its interrupt path

2.5 Deliberate Scope Boundaries

These are decisions, not defects. Each is recorded here because each one looks like a bug to somebody reading the RTL for the first time.

IRQ2 is never driven by the fabric. In a PC/AT pair, master IR2 belongs to the cascade: it carries the slave controller's `INT`, not a device. `rlb_top` masks IR2 off every other source and forces it from the slave, so an interrupt routed to IRQ2 reaches nothing. This is also why IRQ8 exists as a line at all.

The general HPET timers are not routed. `hpet_timer_irq` has no conventional legacy line, which is the same reason the HPET's `TIMER_INT_ROUTE_CAP` field reads 0. Those timers reach `rlb_irq_out` and their own output pins. Only the two legacy-replacement outputs, `hpet_legacy_irq0` and `hpet_legacy_irq8`, enter the fabric.

`ioapic_msi_emit` is not instantiated. Software can program `cfg_msi_addr` and `cfg_msi_data` through the IOAPIC's `IOWIN`, but nothing in this subsystem consumes them. The pins are connected and deliberately left open rather than omitted, because an omitted pin is a `PINMISSING` warning that hides the gap.

Boot-interrupt rerouting has a default map, not a board map. The pin to legacy-input mapping is a board and chipset decision. The default is the published identity mapping for pins 0-7 with no reroute above that, and rerouting is disabled at reset.

Every block is instantiated with `CDC_ENABLE(0)`. The per-block clock and reset ports are present on `rlb_top`'s boundary but unused in this configuration. See Clocks and Reset.

2.6 Applications

- A drop-in legacy peripheral subsystem for a soft-core SoC on FPGA
- A target for firmware and driver development against PC-compatible peripherals without the hardware
- A worked example of an integration level: generated address decode, an interrupt fabric with convention-versus-choice documented, and a cascade

2.7 Related Documents

- Architecture - how the pieces fit together
- Top-Level Interface - the integration contract
- Initialization - bring-up order
- Window Map - the address map in detail

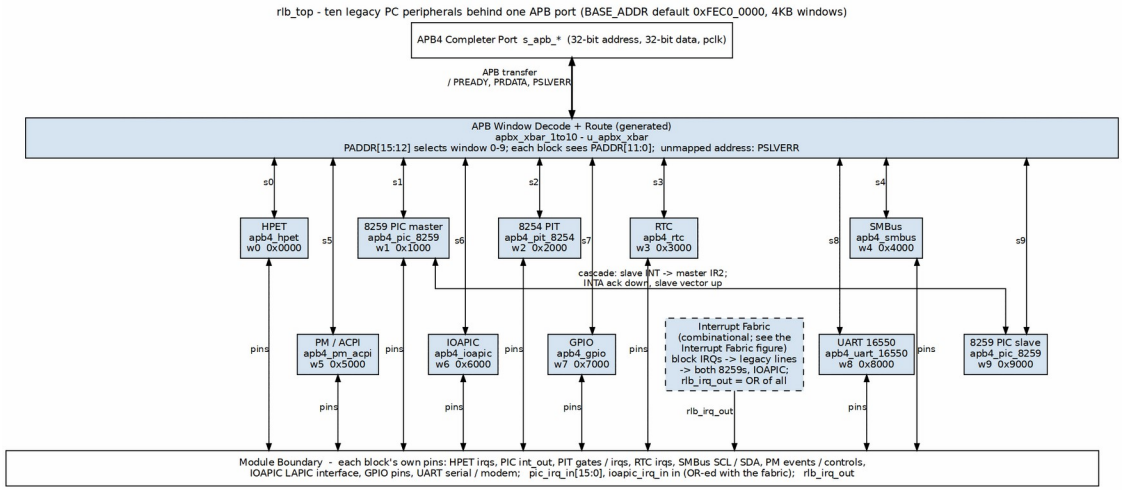
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 RLB Top - Architecture

3.1 Hierarchy

```
rlb_top
├── u_apbx_xbar          apbx_xbar_1to10     one master, ten slaves
(GENERATED)
├── u_hpet               apb4_hpet           window 0
├── u_pic                apb4_pic_8259       window 1   master of the
pair
├── u_pic_slave          apb4_pic_8259       window 9   slave of the
pair
├── u_pit                apb4_pit_8254       window 2
├── u_rtc                apb4_rtc            window 3
├── u_smbus              apb4_smbus          window 4
├── u_pm_acpi            apb4_pm_acpi        window 5
├── u_ioapic             apb4_ioapic         window 6
├── u_gpio               apb4_gpio           window 7
├── u_uart               apb4_uart_16550     window 8
└── u_ioapic_boot_intx   ioapic_boot_intx    no window; combinational
companion
```



RLB Top Integration Hierarchy

Everything else in the module is wiring and two pieces of combinational logic: the interrupt fabric and the cascade cross-connect.

3.2 The Address Path

One APB4 slave port enters the crossbar. The crossbar decodes `PADDR[15:12]` into a slave index and forwards the transaction, including `PSTRB` and `PPROT`, to that slave's port.

The crossbar is generated. It comes from the shared crossbar generator, configured as one master, ten slaves, base `0xFEC00000`, 32-bit address and data, 4KB per slave. Do not hand-edit `apbx_xbar_1to10.sv`. The generator owns the decode-miss path, where an unmapped address completes with `PSLVERR` instead of hanging, and a hand-rolled copy is exactly how that behaviour gets lost - it already happened once to the crossbar this one replaced.

The crossbar routes the full address; each block takes the low 12 bits. Every peripheral instantiation slices `PADDR[11:0]` off its window's channel. A block therefore sees offsets from `0x000` within its own window and needs no knowledge of `BASE_ADDR`. This is the one detail most likely to trip up a reader comparing the top-level port width (32 bits) against a block's (12 bits).

Property	Value
Base address	`BASE_ADDR`, default `0xFEC0_0000`
Window size	4KB (`0x1000`)
Windows	10, indices 0-9, none reserved
Slave index	`PADDR[15:12]`
Address seen by a block	`PADDR[11:0]` of its own window
Unmapped address	completes with `PSLVERR`

3.3 The Interrupt Fabric

Each block emits its interrupt on its own output pin, and those pins remain on the boundary. The fabric additionally collects them into an internal 16-bit vector, `w_fabric_irq`, on the conventional legacy line numbers, and feeds that vector to three destinations.

3.3.1 Source to line

Source signal	Line	Basis	Reaches
`pit_timer_irq[0]` OR `hpet_legacy_irq0`	IRQ0	convention (system timer)	master 8259 IR0, IOAPIC pin 0
`uart_irq`	IRQ4	convention (COM1)	master 8259 IR4, IOAPIC pin 4
`rtc_alarm_irq` OR `rtc_second_irq` OR `hpet_legacy_irq8`	IRQ8	convention (RTC alarm)	slave 8259 IR0, IOAPIC pin 8
`pm_interrupt`	IRQ9	convention (ACPI)	slave 8259 IR1, IOAPIC pin 9
`smb_interrupt`	IRQ10	**choice**, parameter	slave 8259 IR2,

Source signal	Line	Basis	Reaches
		IRQ_SMBUS	IOAPIC pin 10
gpio_irq	IRQ11	**choice**, parameter IRQ_GPIO	slave 8259 IR3, IOAPIC pin 11
hpet_timer_irq[*]	none	no conventional line exists	rlb_irq_out and its own pins only
-	IRQ2	reserved for the cascade	never driven by the fabric

Convention versus choice is a real distinction here, and it is reflected in the RTL. IRQ0, IRQ4, IRQ8 and IRQ9 come from the published typical-assignment table reproduced in the IOAPIC specification, so they are fixed as `localparam` inside the module: making them overridable would invite silently diverging from what every driver already assumes. SMBus and GPIO have no traditional assignment - the table offers only “Available” - so those two are this subsystem’s choice and are exposed as module parameters an integrator can move.

Two sources share IRQ0 and three share IRQ8. That is intentional and safe: HPET legacy-replacement mode exists precisely to replace the 8254 tick and the RTC periodic interrupt, and the HPET core suppresses timers 0 and 1 on `hpet_timer_irq` while that mode is active, so the same event cannot arrive twice.

3.3.2 The three destinations

Master 8259. The master sees

```
w_master_pic_irq = ((pic_irq_in[7:0] | w_boot_intx_pic_irq |
w_fabric_irq[7:0]) & 8'hFB)
                 | {5'b0, w_spic_int, 2'b0}
```

Note the order: the external input, the boot-interrupt reroute and the fabric are OR-ed together first, then `& 8'hFB` clears bit 2, and only then is bit 2 driven from the slave’s `INT`. The mask is what makes the cascade trustworthy - without it an external IRQ2, or a boot-interrupt reroute onto pin 2, could impersonate the slave controller. Every other master level passes through unchanged.

Slave 8259. The slave’s `irq_in` is the inline expression `pic_irq_in[15:8] | w_fabric_irq[15:8]`. It is not a named signal, which is worth knowing when probing: with `pic_irq_in` held at zero, the slave’s IR line for a given IRQ *is* `w_fabric_irq[irq]`.

IOAPIC. `w_ioapic_irq = ioapic_irq_in | w_fabric_irq`, zero-extended to the IOAPIC’s pin count. The fabric defines 16 legacy lines; the IOAPIC has `IOAPIC_NUM_IRQS` pins, default 24, and the upper pins are for additional devices this subsystem does not source.

3.3.3 OR-ed in, never substituted

`pic_irq_in` and `ioapic_irq_in` remain inputs and are OR-ed with the fabric. This is deliberate: an integrator keeps the external interrupt path, and every test that drove those pins directly still works. Replacing them would have been a silent change to the port contract.

`pic_int_out` is **not** a fabric source. Feeding the controller's own output back into its inputs is a combinational loop. It appears only in the aggregate below, which drives nothing internally.

3.4 The Cascade Cross-Connect

The two 8259 instances are wired as a PC/AT pair.

Direction	Signal	Purpose
slave to master	`w_spic_int`	the slave's `INT`, forced onto master IR2
master to slave	`w_pic_cas_ack`	the master's cascade acknowledge
slave to master	`w_spic_vector`	the slave's vector, returned by the master in place of its own

The master's `cas_ack_in` ties low, because nothing acknowledges the master from above. The invariant this creates is that master IR2 always equals `w_spic_int` - not that IR2 never moves. For any IRQ8-15 source, IR2 *should* rise; that is the cascade working.

3.5 The Aggregated Output

`rlb_irq_out` is a single line asserted while any block interrupt is asserted:

```
rlb_irq_out = |{hpet_timer_irq, hpet_legacy_irq0, hpet_legacy_irq8,
                pic_int_out, pit_timer_irq,
                rtc_alarm_irq, rtc_second_irq, smb_interrupt,
                pm_interrupt, gpio_irq, uart_irq}
```

It exists for a system that wants one interrupt input rather than ten pins. It is a pure OR, it includes `pic_int_out` because that is what a single-line consumer wants to see, and it takes no part in the routing above - which is precisely why including the controller's own output is safe here.

3.6 Boot-Interrupt Rerouting

The IOAPIC exports which pins it is *not* delivering. `u_ioapic_boot_intx` turns that into legacy PIC inputs, so a masked IOAPIC pin can still reach the 8259.

The map is the published identity mapping - IOAPIC pin *n* to legacy input *n* for *n* in 0 to 7 - narrowed to the eight inputs the master controller has, with a no-reroute code for pins 8 and above. Pin 2 is a dead entry in that table: it still names legacy input 2, but master IR2 is masked off the external inputs and driven from the slave, so a reroute onto pin 2 reaches nothing. It is left in the map so the table still reads as the identity mapping; an integrator who needs pin 2 rerouted picks a different legacy input.

Rerouting is **safe by default**: the enable bit resets to 0, so `w_boot_intx_pic_irq` is all zeros and the master's OR is exactly `pic_irq_in` until software opts in.

3.7 Related Documents

- Overview
- Clocks and Reset
- Top-Level Interface
- Initialization

4 RLB Top - Clocks and Reset

4.1 The Short Version

There is one functional clock domain in this configuration: `pclk`. Every other clock port exists on the boundary and is unused.

That needs stating plainly, because the port list suggests otherwise.

4.2 Clock and Reset Ports

Clock	Reset	Intended domain	Used in this configuration
`pclk`	`presetn`	APB, primary	**Yes** - everything runs here
`hpet_clk`	`hpet_resetn`	HPET counter	No
`pit_clk`	`pit_resetn`	PIT counting clock	No
`rtc_clk`	`rtc_resetn`	RTC time base	No
`smbus_clk`	`smbus_resetn`	SMBus bit clock	No
`pm_clk`	`pm_resetn`	PM/ACPI	No
`ioapic_clk`	`ioapic_resetn`	IOAPIC	No
`gpio_clk`	`gpio_rstn`	GPIO sampling	No
`uart_clk`	`uart_rstn`	UART baud reference	No

Note the naming inconsistency in the last two: GPIO and UART use `gpio_rstn` and `uart_rstn`, not `_resetn`. That is the blocks' own port naming, carried through.

4.3 Why the Extra Clocks Are Unused

`rlb_top` instantiates every CDC-capable block with `CDC_ENABLE(0)`:

Instance	Parameterisation
`u_hpet`	`CDC_ENABLE(0)`
`u_pit`	`CDC_ENABLE(0)`
`u_smbus`	`CDC_ENABLE(0)`

Instance	Parameterisation
u_pm_acpi	CDC_ENABLE(0)
u_ioapic	CDC_ENABLE(0)
u_gpio	CDC_ENABLE(0)
u_uart	CDC_ENABLE(0)

With CDC_ENABLE=0 a block uses the plain APB slave and ignores its second clock and reset entirely. The ports remain on rlb_top so the boundary does not change if a future configuration enables CDC on one or more blocks, but today nothing downstream of them is clocked.

What an integrator should do: tie every unused clock to pclk and every unused reset to presetn. Leaving them unconnected or tied to a constant is legal in this configuration but becomes a real bug the moment any block is rebuilt with CDC_ENABLE=1, and the failure then looks like a dead peripheral rather than a missing clock.

The RTC is the case worth flagging: a real system usually wants its time base on a separate always-on 32.768 kHz domain, which in this subsystem means instantiating apb4_rtc with CDC enabled rather than relying on rtc_clk as wired here.

4.4 Reset Behaviour

- **Active low, asynchronous**, throughout. There is no positive-polarity reset anywhere in this subsystem.
- Reset is applied through the repository's reset macros from reset_defs.svh, which rlb_top includes. A hand-written always_ff @(posedge clk or negedge rst_n) is not used and would be rejected.
- rlb_top itself holds no state. It has no always_ff of its own: the interrupt fabric is a single always_comb, and the cascade and aggregate are continuous assignments. All reset behaviour belongs to the instantiated blocks.

4.4.1 Reset state of the interrupt fabric

Because the fabric is purely combinational, its reset state is whatever its sources are, and at reset those are all deasserted. Two consequences matter for bring-up:

- **w_boot_intx_pic_irq is all zeros after reset**, because the IOAPIC's boot-interrupt enable resets to 0. The master's interrupt input is therefore exactly pic_irq_in until software enables rerouting.
- **IOAPIC redirection entries reset masked.** A source can reach the IOAPIC's pin and produce no delivery at all, which looks identical to a broken route. Unmasking the relevant entry is a required bring-up step, not an optimisation. See Initialization.

4.4.2 Resetting to clear an interrupt

The 8259 in edge-triggered mode latches `INT` high until it is acknowledged. The integration test suite resets the whole subsystem between phases rather than acknowledging, because the acknowledge and EOI paths have their own known limitations at block level. That is a verification convenience and not a recommendation for firmware; software should acknowledge properly. It is mentioned here because it explains why the testbench re-programs both controllers and re-arms the IOAPIC after every such phase - a full reset drops all of that configuration.

4.5 Related Documents

- Architecture
- Top-Level Interface
- Repository reset and clocking rules: `vault/handbook/design/reset-and-clocking.md`

5 RLB Top - Acronyms and Terminology

5.1 Acronyms

Term	Expansion	In this subsystem
ACPI	Advanced Configuration and Power Interface	The power-management block on window 5
APB	Advanced Peripheral Bus	The bus every block speaks; APB4 here
APB4	AMBA 4 APB	The revision in use, with `PSTRB` and `PPROT`
CDC	Clock Domain Crossing	Parameter `CDC_ENABLE`, 0 in this configuration
EOI	End Of Interrupt	Acknowledge written back to a controller
GPE	General Purpose Event	ACPI wake and status events, `pm_gpe_events`
GPIO	General Purpose Input/Output	The controller on window 7
HPET	High Precision Event Timer	The timer on window 0
ICW	Initialization Command Word	8259 configuration writes, ICW1-ICW4
IOAPIC	I/O Advanced Programmable Interrupt Controller	The controller on window 6
IR	Interrupt Request line	An individual input of an 8259, IR0-IR7
IRQ	Interrupt Request	A legacy line number, IRQ0-IRQ15
MSI	Message Signalled Interrupt	Configurable in the IOAPIC, not emitted here
OCW	Operation Command Word	8259 runtime writes, OCW1-OCW3
PIC	Programmable Interrupt	The 8259 pair, windows

Term	Expansion	In this subsystem
	Controller	1 and 9
PIT	Programmable Interval Timer	The 8254 on window 2
PM	Power Management	Used interchangeably with ACPI here
RLB	Retro Legacy Blocks	This subsystem
RTC	Real-Time Clock	The block on window 3
RTE	Redirection Table Entry	An IOAPIC per-pin configuration entry
SMBus	System Management Bus	The controller on window 4
UART	Universal Asynchronous Receiver/Transmitter	The 16550 on window 8

5.2 Terminology

Window. One of the ten 4KB address regions behind the crossbar. Window *n* starts at `BASE_ADDR + n * 0x1000`. The window number equals the crossbar's slave index, which equals `PADDR[15:12]`.

Fabric. The internal interrupt routing in `rlb_top`: the `w_fabric_irq` vector and the logic that builds it and distributes it. Not a bus.

Line. A legacy IRQ number, 0 to 15. A line is a position in the fabric vector, and separately an input on one of the two controllers.

Convention versus choice. A line assignment is *convention* if it comes from the published typical-assignment table, and a *choice* if this subsystem picked it because no traditional assignment exists. Convention assignments are fixed in the RTL; choices are module parameters.

Master and slave. The two 8259 instances. The master is on window 1 and drives `pic_int_out`; the slave is on window 9 and reaches the CPU only through the master's IR2. Unrelated to the APB sense of master and slave.

Cascade. The connection between the two controllers. Master IR2 carries the slave's `INT`, the master's acknowledge reaches the slave, and the slave's vector is returned in place of the master's.

Boot interrupt. A source whose IOAPIC pin is masked, rerouted to the legacy controller so it is not lost during boot before the IOAPIC is configured.

Legacy replacement. An HPET mode in which HPET timers 0 and 1 take over the 8254 tick and the RTC periodic interrupt, appearing on `hpet_legacy_irq0` and `hpet_legacy_irq8` while being suppressed on `hpet_timer_irq`.

5.3 Related Documents

- Overview
- References

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 RLB Top - References

6.1 External Specifications

Every peripheral in this subsystem implements a published specification. Copies of the specifications used are kept in the component's `References/` directory.

Block	Specification
HPET	High Precision Event Timer specification, revision 1.0a
8259 PIC	8259A programmable interrupt controller datasheet
8254 PIT	82C54 interval timer datasheet
RTC	MC146818 real-time clock plus RAM datasheet
SMBus	System Management Bus specification, version 3.3
PM/ACPI	Advanced Configuration and Power Interface specification
IOAPIC	I/O advanced programmable interrupt controller datasheet
UART 16550	16550 asynchronous communications element datasheet
Bus	AMBA APB protocol specification, APB4

The interrupt line assignments this subsystem treats as convention - IRQ0 for the system timer, IRQ4 for COM1, IRQ8 for the RTC alarm and IRQ9 for ACPI - come from the typical-assignment table reproduced in ioapic_mas ch05. That table is the authority cited in the RTL, and it is also the evidence that IRQ10 and IRQ11 have no traditional assignment, which is why SMBus and GPIO are parameters rather than fixed.

6.2 In-Repository References

6.2.1 This subsystem

Item	Location
Integration RTL	rtl/rlb_top/rlb_top.sv
Integration filelist	rtl/rlb_top/filelists/rlb_top.f
Generated crossbar	rtl/apbx_xbar/apbx_xbar_1to10.sv
Test runner	dv/tests/test_rlb_top.py
Testbench classes	dv/tbclasses/rlb_top/
Subsystem requirements	PRD.md
Subsystem overview	README.md
Area facts and traps	CLAUDE.md
FPGA integration guide	docs/RLB_FPGA_IMPLEMENTATION_GUIDE.md
Implementation status	docs/IMPLEMENTATION_STATUS.md
Driver development reading list	docs/LegacyBlocksAndDriverGuide.md

6.2.2 Per-block specifications

Block	Specification
HPET	hpet_mas
8259 PIC	pic_8259_mas
8254 PIT	pit_8254_mas
RTC	rtc_mas
SMBus	smbus_mas
PM/ACPI	pm_acpi_mas
IOAPIC	ioapic_mas
GPIO	gpio_mas
UART 16550	uart_16550_mas

6.2.3 Repository practice

The methods this subsystem follows are recorded in the handbook rather than restated here:

Topic	Note
Generated RTL discipline	`vault/handbook/design/generated-rtl-discipline.md`
Reset and clocking	`vault/handbook/design/reset-and-clocking.md`
Filelists	`vault/handbook/design/filelists.md`
Running regressions	`vault/handbook/dv/running-regressions.md`
Register map links	`vault/handbook/authoring/register-map-links.md`

6.3 Related Documents

- Overview
- Acronyms

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 RLB Top - Block Hierarchy Overview

7.1 The Twelve Instances

`rlb_top` contains no logic blocks of its own. It instantiates twelve modules and wires them together, with two pieces of combinational glue: the interrupt fabric and the cascade cross-connect.

Instance	Module	Window	Parameters passed
`u_apbx_xbar`	`apbx_xbar_1to10`	-	`ADDR_WIDTH`, `DATA_WIDTH`, `STRB_WIDTH`, `BASE_ADDR`
`u_hpet`	`apb4_hpet`	0	`VENDOR_ID`, `REVISION_ID`, `NUM_TIMERS`, `CDC_ENABLE(0)`
`u_pic`	`apb4_pic_8259`	1	none
`u_pit`	`apb4_pit_8254`	2	`NUM_COUNTERS`, `CDC_ENABLE(0)`
`u_rtc`	`apb4_rtc`	3	none
`u_smbus`	`apb4_smbus`	4	`FIFO_DEPTH`, `CDC_ENABLE(0)`
`u_pm_acpi`	`apb4_pm_acpi`	5	`CDC_ENABLE(0)`
`u_ioapic`	`apb4_ioapic`	6	`NUM_IRQS`, `CDC_ENABLE(0)`
`u_gpio`	`apb4_gpio`	7	`GPIO_WIDTH`, `SYNC_STAGES`, `CDC_ENABLE(0)`, `SKID_DEPTH(2)`
`u_uart`	`apb4_uart_16550`	8	`FIFO_DEPTH`, `SYNC_STAGES`, `CDC_ENABLE(0)`, `SKID_DEPTH(2)`
`u_pic_slave`	`apb4_pic_8259`	9	none
`u_ioapic_boot_intx`	`ioapic_boot_intx`	-	`NUM_IRQS`, `NUM_PIC(8)`, `PIC_IDX_W(4)`,

Instance	Module	Window	Parameters passed
			PIC_MAP

7.2 u_apbx_xbar - the address decode

A generated 1-to-10 APB crossbar. It is produced by the shared crossbar generator, configured as one master, ten slaves, base 0xFEC00000, 32-bit address and data, and 4KB per slave.

Do not hand-edit it. The generator owns the decode-miss path - an unmapped address completes with PSLVERR rather than hanging the bus - and the hand-rolled crossbar this one replaced had already lost that behaviour. It is regenerated, not patched.

The crossbar forwards the full 32-bit address. Each peripheral instantiation slices PADDR[11:0] from its own channel, so a block sees offsets from 0x000 within its window.

7.3 The two 8259 instances

The same module appears twice, and the difference is entirely in the wiring.

	u_pic (master, window 1)	u_pic_slave (slave, window 9)
irq_in	w_master_pic_irq (a named signal)	pic_irq_in[15:8] | w_fabric_irq[15:8] (inline)
int_out	pic_int_out, a module output	w_spic_int, internal only
cas_vector	w_spic_vector, from the slave	8'h00, tied off
cas_ack_in	1'b0 - nothing acknowledges the master	w_pic_cas_ack, from the master
cas_ack	w_pic_cas_ack, to the slave	unconnected
inta_vector_o	unconnected	w_spic_vector, to the master

Two details are worth carrying away. First, the slave's irq_in is an **inline expression, not a named signal**, so there is nothing to probe by name; with pic_irq_in held at zero the slave's IR line for a given IRQ is exactly w_fabric_irq[irq]. Second, the master's IR2 is not OR-ed with anything - it is masked off every other source and then forced from the slave.

7.4 u_ioapic - and what it is not wired to

The IOAPIC receives `w_ioapic_irq`, which is the external `ioapic_irq_in` OR-ed with the zero-extended fabric vector. Its delivery handshake, EOI input and retry input are all on the module boundary.

Four of its configuration outputs are **connected and deliberately left open**:

Pin	Why it is open
`cfg_msi_addr`	`ioapic_msi_emit` is not instantiated here, so nothing consumes MSI configuration
`cfg_msi_data`	as above
`cfg_mask_vec`	consumed by `u_ioapic_boot_intx`
`cfg_boot_intx_en`	consumed by `u_ioapic_boot_intx`

The first two are the interesting case. Software can program them through `IOWIN`, and nothing will happen. They are wired to an empty connection rather than omitted from the instantiation because an omitted pin produces a `PINMISSING` warning, which is how a gap like this becomes invisible.

7.5 u_ioapic_boot_intx - the boot-interrupt companion

A small combinational module. It takes the raw IOAPIC pin inputs, the IOAPIC's mask vector and its boot-interrupt enable, and produces legacy PIC inputs for the pins the IOAPIC is not delivering.

Its map, `BOOT_INTX_PIC_MAP`, is built as a localparam in `rlb_top`: the identity mapping for pins 0 through 7, and a no-reroute code for every pin above that. Field 0 is the least significant, so the identity entries are written last in the concatenation.

Pin 2 is a **dead entry**. The map still names legacy input 2, but master IR2 is masked off the external inputs and driven from the slave controller, so a reroute onto pin 2 reaches nothing. It is kept in the map so the table still reads as the published identity mapping; an integrator who needs pin 2 rerouted must choose a different legacy input.

Because the IOAPIC's boot-interrupt enable resets to 0, this module's output is all zeros after reset and the master's interrupt input is exactly `pic_irq_in` until software opts in.

7.6 The glue

Neither of these is a module, and both are worth naming because they are where the integration behaviour lives.

The interrupt fabric is one `always_comb` block that builds `w_fabric_irq[15:0]` from the blocks' interrupt outputs, plus three continuous assignments that distribute it: to the IOAPIC, to the master controller with the IR2 mask, and into the `rlb_irq_out` aggregate.

The cascade cross-connect is three wires between the two controllers, listed in the table above.

`rlb_top` has no `always_ff` of its own. It holds no state.

7.7 Related Documents

- Architecture - the fabric and cascade in detail
- Top-Level Interface - the port list
- Window Map

8 RLB Top - The APB Crossbar Block

8.1 Overview

`u_apbx_xbar` is the only address decoder in the subsystem. Every APB transaction that enters on `s_apb_*` is steered to exactly one of ten peripheral channels here, and every decode decision a debugger needs to make — *which block is this address in?* — is answered by this module's window map and nothing else.

It is not hand-written. It is generated by the shared crossbar generator and treated as a build artifact: regenerated when the map changes, never patched.

8.2 Generation

Produced by `projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/apbx_xbar_generator.py`:

```
generate_apbx_xbar(num_masters=1, num_slaves=10, base_addr=0xFEC00000,
                   addr_width=32, data_width=32, slave_size=0x1000)
```

`rlb_top` instantiates the result as `apbx_xbar_1to10` and passes only the global bus parameters through: `ADDR_WIDTH`, `DATA_WIDTH`, `STRB_WIDTH`, `BASE_ADDR`. The slave count and window size are baked into the generated RTL, not parameterised at the instantiation.

8.3 Slave Index to Window

The generated module names its peripheral channels `s0_apb_*` through `s9_apb_*`. The mapping to the ten 4KB windows is fixed at generation time:

Slave	Window	Block	Base offset
s0	0	HPET	`0x0000`
s1	1	Master 8259 PIC	`0x1000`
s2	2	8254 PIT	`0x2000`
s3	3	RTC	`0x3000`
s4	4	SMBus	`0x4000`
s5	5	PM/ACPI	`0x5000`
s6	6	IOAPIC	`0x6000`
s7	7	GPIO	`0x7000`
s8	8	UART 16550	`0x8000`

Slave	Window	Block	Base offset
s9	9	Slave 8259 PIC	`0x9000`

The full decode table, including probe registers for each window, lives in Chapter 5 and Chapter 3.1. This page owns the *mechanism*; those pages own the *map*.

Two facts about the map are worth recording where the crossbar is described:

- **There is no reserved slave.** Window 9 decoded as "Reserved" when the crossbar was generated. When the slave 8259 moved into that window (RLB/pic_8259 TASK-001) the decode already existed — only the wires on `s9_apb_*` changed owner. No regeneration was needed, and no reserved window remains in the address map.
- **The window number is the address nibble.** With 4KB windows, the slave select is exactly `PADDR[15:12]` above `BASE_ADDR`, so the window number, the slave index, and the address nibble are the same value.

8.4 Address Forwarding

The crossbar forwards the **full 32-bit address** on each slave channel. Each peripheral instantiation slices `PADDR[11:0]` from its own channel, so every block sees register offsets starting at `0x000` within its own window and never learns the absolute address. A block that needs no more than 12 bits of offset is therefore insulated from the base address; only `rlb_top` knows `BASE_ADDR` (default `0xFEC00000`, a parameter).

8.5 Decode Miss Behaviour

An address that matches no slave completes with `PSLVERR` asserted instead of hanging the bus. This behaviour is owned by the **generator**, not by this instance: the generator emits the decode-miss path, and the hand-rolled crossbar this one replaced had already lost it. That history is why the rule is *regenerate, don't edit* — a patch on the generated file is exactly how the fix disappears. The unmapped-address error response is exercised by the `func` suite (see the book index, "What the suite proves", item 4).

8.6 What This Block Does Not Do

- **No interrupt involvement.** The crossbar is a pure address/data path. Interrupts route around it entirely — see The Interrupt Fabric.
- **No clock or reset generation.** It runs on `pclk` / `presetn` like every block in this configuration; all `CDC_ENABLE(0)`.
- **No slave-side wait-state merging policy** beyond what the generator emits: each peripheral's `PREADY` is returned on its own window.

8.7 Related Documents

- Block Hierarchy Overview - the twelve instances
- Top-Level Interface - the address decode contract this block implements
- Window Map - the decode table

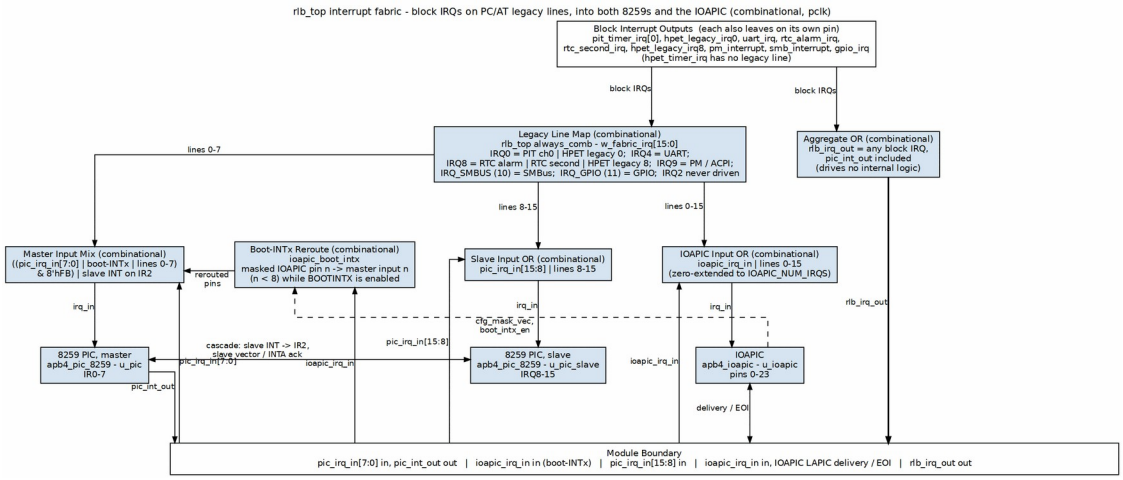
---

9 RLB Top - The Interrupt Fabric Block

9.1 Overview

The interrupt fabric is not a module. It is one `always_comb` block that builds `w_fabric_irq[15:0]` from the blocks' interrupt outputs, plus three continuous assignments that distribute the result: to the two 8259s, to the IOAPIC, and into the `rlb_irq_out` aggregate. It is listed here as a block because the integration behaviour lives in this glue, and a reader who skips it understands the address map but not the subsystem.

Its job: put every block interrupt on its **conventional legacy line** so that standard driver software and either interrupt controller see a recognisable PC/AT wiring, without the board having to wire ten interrupt pins back in.



RLB Top Interrupt Fabric

9.2 The Routing Table

Seven block-level sources drive the fabric's six lines — PIT channel 0 and HPET's legacy pair both land on IRQ0. The six driven lines:

Line	Driver(s)	Origin
IRQ0	`pit_timer_irq[0]` | `hpet_legacy_irq0`	8254 channel-0 tick, replaced by HPET timer 0 in legacy-replacement mode
IRQ4	`uart_irq`	UART 16550 (COM1 by convention)
IRQ8	`rtc_alarm_irq` |	RTC alarm + periodic,

Line	Driver(s)	Origin
	`rtc_second_irq` | `hpet_legacy_irq8`	replaced by HPET timer 1 in legacy mode
IRQ9	`pm_interrupt`	PM/ACPI
IRQ10	`smb_interrupt`	SMBus — **choice**, parameter `IRQ_SMBUS`
IRQ11	`gpio_irq`	GPIO — **choice**, parameter `IRQ_GPIO`

Every other fabric line is constant zero, including **IRQ2, which is deliberately never driven**: it is the 8259 cascade input, carried from the slave controller's `INT` (see The Cascade Cross-Connect). Anything routed onto IRQ2 reaches nothing — QEMU shipped exactly that bug, which is why the line is commented in the RTL rather than left implicit.

9.2.1 Convention versus choice

IRQ0/4/8/9 come straight from the "Typical IRQ Assignments" table in the IOAPIC book's register chapter, and are fixed `localparam`s inside `rlb_top`. Making them overridable would invite silently diverging from the wiring every driver already assumes. SMBus and GPIO have no traditional line — the table offers only "Available" — so those two are this subsystem's choice and are exposed as module parameters, meant to be changed by an integrator who needs them elsewhere.

9.2.2 The HPET legacy pair

`hpet_legacy_irq0` and `hpet_legacy_irq8` are not the general `hpet_timer_irq` outputs. They are asserted only while `HPET_CONFIG.legacy_replacement` is set, and while it is set the HPET core suppresses timers 0/1 on their own pins — so the HPET and the device it replaces can never both drive the same line for the same event. The general `hpet_timer_irq` lines are deliberately **not** routed into the fabric: they have no conventional legacy line (the same reason `TIMER_INT_ROUTE_CAP` reads 0), and they reach `rlb_irq_out` and their own pins only.

9.3 Distribution

The built vector goes to three consumers:

Both 8259s. The master sees the low half through the IR2 mask and cascade force described in the cascade page; the slave OR-es the upper half into its external input inline: `pic_irq_in[15:8] | w_fabric_irq[15:8]`. The fabric is **OR-ed into the external inputs, not substituted for them** — `pic_irq_in` stays an input, so a board keeps its external interrupt path and every existing test keeps its direct drive.

The IOAPIC. `w_ioapic_irq = ioapic_irq_in | {zero-extend of w_fabric_irq}` — the same sources on the same pin numbers, zero-extended to `IOAPIC_NUM_IRQS` (24). The upper eight pins are PCI or additional devices this subsystem does not source; they remain for the board.

The aggregate. `rlb_irq_out` is a pure OR of every block interrupt, including `pic_int_out` — the delivered controller output, which is exactly what a single-line consumer wants. `pic_int_out` appears **only** here: feeding the 8259's own output back into its inputs would be a combinational loop, and the aggregate drives no internal logic, so it is safe to include.

9.4 What the Fabric Does Not Do

- **No masking, latching, or priority.** It is wires and one OR tree. All policy — masking, vector selection, delivery — belongs to the 8259s and the IOAPIC, and to the software that programs them (Chapter 4).
- **No CDC.** The fabric is combinational in the `pclk` domain. Every block runs `CDC_ENABLE(0)` in this configuration, so block interrupts are already synchronous to `pclk` before they reach the fabric.
- **No boot-interrupt rerouting.** That is the separate `u_ioapic_boot_intx` companion, described in the Block Hierarchy Overview — it produces its own PIC inputs, which the master's input OR then combines with the fabric's.

9.5 Verification Heritage

The routing table and the "OR-ed, not substituted" contract are the subject of RLB TASK-015 (the fabric), TASK-017/TASK-018 (per-line and per-block routing proof) and TASK-019 (the follow-up batch). The `full` suite proves each of the six sourcing blocks reaches its own master or slave IR line, per line rather than in aggregate, and its IOAPIC pin, and passes coincident-assert cases across both PICs.

9.6 Related Documents

- Block Hierarchy Overview - where the fabric sits among the twelve instances
- The Cascade Cross-Connect - why IRQ2 is never driven
- Per-Block Interrupt Pin Reference - pin-level table including the pins the fabric does not touch
- Use Cases - routing an interrupt end to end

10 RLB Top - The 8259 Cascade Cross-Connect

10.1 Overview

The subsystem carries a PC/AT interrupt pair: `u_pic` is the master (window 1), `u_pic_slave` is the slave (window 9), and the two are wired together inside `rlb_top` exactly as the cascaded configuration the 8259 was designed for. The cross-connect between them is three named signals — the only internal state path `rlb_top` owns besides the interrupt fabric. This page is the wiring in detail; the software sequences that bring the pair up are in Chapter 4.1.

10.2 Why the Pair Exists At All

Sixteen interrupt inputs need two 8259s: the master covers IRQ0-7, the slave covers IRQ8-15. The slave's window at 0xFEC09000 is what makes IRQ8-15 exist — and IRQ8 in particular is the line HPET timer 1 replaces when legacy-replacement mode is on (RLB/hpet TASK-003).

10.3 The Three-Wire Cross-Connect

Signal	From	To	Carries
`w_spic_int`	slave `int_out`	master IR2 (forced)	The slave's interrupt request
`w_pic_cas_ack`	master `cas_ack`	slave `cas_ack_in`	The cascade acknowledge
`w_spic_vector`	slave `inta_vector_o`	master `cas_vector`	The slave's vector, returned by the master in place of its own

The remaining cascade pins are tied or left open by role: the master is `cas_ack_in = 1'b0` because nothing above it acknowledges its INT, and its `inta_vector_o` is unconnected because nobody above consumes the vector. The slave's `cas_vector` input is `8'h00` — it never forwards a deeper cascade — and its `cas_ack` output is unconnected.

10.4 The Master: IR2 Is Forced, Not OR-ed

The master's interrupt input is:

```
assign w_master_pic_irq = ((pic_irq_in[7:0] | w_boot_intx_pic_irq
                          | w_fabric_irq[7:0]) & 8'hFB)
                        | {5'b0, w_spic_int, 2'b0};
```

Reading it inside out: the external inputs, the boot-interrupt reroute, and the fabric's low half are OR-ed together, then **bit 2 is masked off** (`& 8'hFB`), and then the slave's `INT` is forced onto that

bit. In a PC/AT pair, master IR2 belongs to the cascade — it carries the slave's INT, not an external device — so an external IRQ2, or a boot-interrupt reroute onto pin 2, would otherwise impersonate the slave. Mask-then-force makes that impossible: every other master level is unchanged, and IR2 is exactly the slave's request.

This is also why **IRQ2 reaches nothing from the outside**. The address map and the interrupt map both still show the line; it is consumed internally by construction.

10.5 The Slave: An Inline Input Worth Naming

The slave's interrupt input is an inline expression, not a named signal:

```
.irq_in (pic_irq_in[15:8] | w_fabric_irq[15:8]),
```

There is nothing to probe by name. With `pic_irq_in` held at zero, the slave's IR line for a given IRQ is exactly `w_fabric_irq[irq]` — which is how the `full` suite verifies the slave-routed sources per line: drive the block, watch `w_fabric_irq[15:8]`, and the slave's view is known by construction.

On the cascade side the slave exports its vector upward (`inta_vector_o` → `w_spic_vector` → master's `cas_vector`), so when the master acknowledges a slave-sourced interrupt, the vector the CPU eventually reads is the slave's — the master returns it in place of its own. That vector path is the entire reason `w_spic_vector` exists as a named wire rather than a port-to-port connection.

10.6 The Boot-Interrupt Side Effect

`u_ioapic_boot_intx` can reroute IOAPIC pins onto legacy PIC inputs, and its identity map still names legacy input 2 for pin 2. That entry has been **dead since the cascade took IR2**: a reroute onto pin 2 is masked off the master's input before the slave's INT is forced on, so it reaches nothing. The map keeps the entry so it still reads as the published 82093AA identity mapping; an integrator who genuinely needs pin 2 rerouted must choose a different legacy input via `PIC_MAP`.

10.7 Verification Heritage

The cross-connect is the subject of RLB/pic_8259 TASK-001. The `func` suite proves the slave answers on window 9 and that the aggregated IRQ reaches `rlb_irq_out`; the `full` suite adds the cascade invariant — a slave-routed source delivered end to end through the master's `int_out` — and coincident asserts spanning both controllers. The bring-up sequences (single controller, cascaded pair) transcribed into Chapter 4.1 come from the cocotb helpers `init_pic` and `init_pic_cascade`, which are known to work because this suite passes.

10.8 Related Documents

- Block Hierarchy Overview - the two-instance wiring table
- The Interrupt Fabric - what reaches the slave's IR lines

- Initialization - programming the pair (ICW3 master/slave values)
- Window Map - window 9 decode

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 RLB Top - Top-Level Interface

11.1 Overview

`rlb_top` is the integration contract for the whole subsystem: one APB4 slave port, the peripherals' external pins, and one aggregated interrupt output. This chapter is the pin list, the parameters, and the address decode rules an integrator has to honour.

11.2 Parameters

Parameter	Type	Default	Description
`ADDR_WIDTH`	int	32	APB address width. The subsystem decodes a full system address, not a windowed one.
`DATA_WIDTH`	int	32	APB data width.
`STRB_WIDTH`	int	`DATA_WIDTH/8`	Write-strobe width. Derived; do not override independently.
`BASE_ADDR`	logic [ADDR_WIDTH-1:0]	`32'hFEC00000`	Base of the ten-window region.
`HPET_VENDOR_ID`	int	1	Driven into the HPET capability register.
`HPET_REVISION`	int	1	Driven into the HPET capability register.
`HPET_NUM_TIMERS`	int	2	HPET comparator count. Sets the width of `hpet_timer_irq`.
`PIT_NUM_COUNTERS`	int	3	8254 counter

Parameter	Type	Default	Description
			count. Sets the width of `pit_gate_in` and `pit_timer_irq`.
`IOAPIC_NUM_IRQS`	int	24	IOAPIC pin count. Sets the width of `ioapic_irq_in`.
`GPIO_WIDTH`	int	32	GPIO pin count.
`GPIO_SYNC_STAGES`	int	2	GPIO input synchroniser depth.
`UART_FIFO_DEPTH`	int	16	UART FIFO depth.
`UART_SYNC_STAGES`	int	2	UART input synchroniser depth.
`SMBUS_FIFO_DEPTH`	int	32	SMBus FIFO depth.
`IRQ_SMBUS`	int	10	**Choice, not convention.** The legacy line SMBus is routed to.
`IRQ_GPIO`	int	11	**Choice, not convention.** The legacy line GPIO is routed to.

11.2.1 Why only two interrupt lines are parameters

IRQ0, IRQ4, IRQ8 and IRQ9 are fixed as `localparam` inside the module because they come from the published typical-assignment table: the system timer, COM1, the RTC alarm and ACPI respectively. Exposing them would invite silently diverging from what every driver already assumes.

SMBus and GPIO have no traditional assignment - the table lists IRQ10 and IRQ11 as available - so those two are this subsystem's choice and are parameters an integrator can move.

11.3 Ports

11.3.1 Clock and Reset

Port	Dir	Width	Notes
pclk	in	1	APB clock. The only functional clock in this configuration.
presetn	in	1	APB reset, active low, asynchronous.
hpet_clk, hpet_resetn	in	1 each	Unused: u_hpet is CDC_ENABLE(0). Tie to pclk / presetn.
pit_clk, pit_resetn	in	1 each	Unused: CDC_ENABLE(0). Tie to pclk / presetn.
rtc_clk, rtc_resetn	in	1 each	Unused in this configuration. Tie to pclk / presetn.
smbus_clk, smbus_resetn	in	1 each	Unused: CDC_ENABLE(0). Tie to pclk / presetn.
pm_clk, pm_resetn	in	1 each	Unused: CDC_ENABLE(0). Tie to pclk / presetn.
ioapic_clk, ioapic_resetn	in	1 each	Unused: CDC_ENABLE(0). Tie to pclk / presetn.
gpio_clk, gpio_rstn	in	1 each	Unused: CDC_ENABLE(0).

Port	Dir	Width	Notes
			Note `_rstn`, not `_resetn`.
`uart_clk`, `uart_rstn`	in	1 each	Unused: `CDC_ENABLE(0)`. Note `_rstn`, not `_resetn`.

See Clocks and Reset for why the secondary clocks are present but unused, and what to do about it.

11.3.2 APB4 Slave Interface

The single entry point to the subsystem.

Port	Dir	Width	Notes
`s_apb_PSEL`	in	1	
`s_apb_PENABLE`	in	1	
`s_apb_PADDR`	in	`ADDR_WIDTH`	**Full system address**, not a window offset.
`s_apb_PWRITE`	in	1	
`s_apb_PWDATA`	in	`DATA_WIDTH`	
`s_apb_PSTRB`	in	`STRB_WIDTH`	Carried through to the selected block.
`s_apb_PPROT`	in	3	Carried through to the selected block.
`s_apb_PRDATA`	out	`DATA_WIDTH`	
`s_apb_PSLVERR`	out	1	Also the decode-miss response.
`s_apb_PREADY`	out	1	

11.3.3 Peripheral External Interfaces

Block	Inputs	Outputs
HPET	-	`hpet_timer_irq[HPET_N`

Block	Inputs	Outputs
		UM_TIMERS-1:0], hpet_legacy_irq0, hpet_legacy_irq8
8259 PIC	pic_irq_in[15:0]	pic_int_out
8254 PIT	pit_gate_in[PIT_NUM_COUNTERS-1:0]	pit_timer_irq[PIT_NUM_COUNTERS-1:0]
RTC	-	rtc_alarm_irq, rtc_second_irq
SMBus	smb_scl_i, smb_sda_i	smb_scl_o, smb_scl_t, smb_sda_o, smb_sda_t, smb_interrupt
PM/ACPI	pm_gpe_events[31:0], pm_gpe1_events[31:0], pm_power_button_n, pm_sleep_button_n, pm_rtc_alarm, pm_ext_wake_n, pm_wdt_reset_n, pm_ext_reset_n, pm_power_domain_ack[7:0]	pm_clock_gate_en[31:0], pm_power_domain_en[7:0], pm_sys_reset_req, pm_periph_reset_req, pm_interrupt
IOAPIC	ioapic_irq_in[IOAPIC_NUM_IRQS-1:0], ioapic_irq_out_ready, ioapic_irq_out_retry, ioapic_eoi_in, ioapic_eoi_vector[7:0]	ioapic_irq_out_valid, ioapic_irq_out_vector[7:0], ioapic_irq_out_dest[7:0], ioapic_irq_out_deliv_mode[2:0], ioapic_irq_out_dest_mode
GPIO	gpio_in[GPIO_WIDTH-1:0]	gpio_out[GPIO_WIDTH-1:0], gpio_oe[GPIO_WIDTH-1:0], gpio_irq
UART 16550	uart_rx, uart_cts_n, uart_dsr_n, uart_ri_n, uart_dcd_n	uart_tx, uart_dtr_n, uart_rts_n, uart_rxrdy_n, uart_txrdy_n,

Block	Inputs	Outputs
Subsystem	-	uart_out1_n, uart_out2_n, uart_irq rlb_irq_out

11.3.4 Ports that need care

Port	What to know
pic_irq_in[2]	**Ignored.** Master IR2 is the cascade input, forced from the slave controller. Driving this bit reaches nothing.
pic_irq_in[15:8]	Goes to the **slave** controller, not the master.
ioapic_irq_out_retry	Tie low if the receiver always accepts. It is the IOAPIC's half of delegated lowest-priority delivery.
pm_power_domain_ack[7:0]	Tie high unless the rail sequencer is enabled with an acknowledge requirement.
pm_wdt_reset_n, pm_ext_reset_n	Tie high (inactive) if the system has no watchdog or reset button. They are reported in the PM reset-status register.
pm_gpe1_events[31:0]	Tie to zero when the system has only one GPE bank.
uart_rxrdy_n, uart_txrdy_n	DMA handshake pins. Leave unconnected in a programmed-I/O system.
hpet_timer_irq	Not routed into the interrupt fabric. Wire it yourself if you need it delivered.
rlb_irq_out	A pure OR of every block interrupt including pic_int_out. Drives nothing internally.

11.4 Address Decode Contract

Property	Value
Base	`BASE_ADDR`, default `0xFEC0_0000`
Window size	`0x1000` (4KB)
Window count	10, indices 0-9
Slave index	`PADDR[15:12]`
Offset seen by a block	`PADDR[11:0]`
Reserved windows	none - all ten are assigned
Unmapped address	completes normally with `PSLVERR` asserted; it does not hang

The decode-miss behaviour is a property of the generated crossbar, and it is the reason the crossbar must be regenerated rather than hand-edited.

11.5 Integration Checklist

1. Drive `pclk` and `presetn`. Tie every secondary clock to `pclk` and every secondary reset to `presetn` unless you have rebuilt a block with `CDC_ENABLE=1`.
2. Connect the APB4 slave port, passing the **full** system address.
3. Decide whether you want the internal interrupt fabric, the external pins, or both. Both is the default and costs nothing: they are OR-ed.
4. If you are not driving external legacy interrupts, tie `pic_irq_in` and `ioapic_irq_in` to zero. The fabric then supplies everything.
5. Tie off the PM pins listed above according to what your system actually has.
6. Choose `IRQ_SMBUS` and `IRQ_GPIO` if the defaults clash with your platform.
7. Decide how the IOAPIC's delivery handshake is consumed, and tie `ioapic_irq_out_retry` low if your receiver always accepts.
8. Use `rlb_irq_out` if you want a single interrupt input; otherwise use `pic_int_out` and the IOAPIC delivery interface.

11.6 Related Documents

- Architecture
- Clocks and Reset
- Initialization
- Window Map

12 RLB Top - APB4 Protocol at the Subsystem Boundary

12.1 Overview

This page is the protocol contract on the single APB slave port `s_apb_*` — the boundary a SoC fabric must honour to talk to the subsystem. The signal list and the address decode table live in Top-Level Interface; this page owns the *rules*: phases, wait states, strobes, protection attributes, and the two ways a transaction can complete with an error.

The boundary is the master side of the generated crossbar (Chapter 2.1). Everything below is a property of that generated fabric plus the blocks behind it, not of any hand-written glue in `rlb_top`.

12.2 Protocol Feature Support

Feature	Support
`PSEL` / `PENABLE` / `PWRITE`	Yes — standard two-phase APB
`PADDR`	32-bit, **full system address** (not a window offset)
`PWDATA` / `PRDATA`	32-bit
`PREADY`	Yes — wait states come from the addressed block (see below)
`PSLVERR`	Yes — two error sources (see below)
`PSTRB`	Yes — carried through to the selected block
`PPROT`	Yes — carried through to the selected block; checking is a per-block decision

The interface is APB4. A master that speaks only APB3 can still use it: tie `PSTRB` to all-ones on writes and all-zero on reads (the APB4 rule — `PSTRB` must be zero during a read), and tie `PPROT` to zero.

12.3 Transfer Phases

A transaction is the standard APB pair:

1. **Setup.** The master asserts `PSEL`, presents the full system address on `PADDR`, the direction on `PWRITE`, and (for writes) `PWDATA`, `PSTRB`, `PPROT`.

2. **Access.** The master asserts PENABLE. The transfer completes on the rising edge where the slave's PREADY is high; read data is sampled on PRDATA in the same cycle.

The minimum transaction length is therefore two pclk cycles. There is no back-to-back packing and no burst concept — APB4 is single-transfer, and the crossbar does not add one.

12.4 Wait States

PREADY returned to the master is the selected block's own PREADY. The fabric between the boundary and the block is combinational: it adds no wait states of its own, so the wait-state profile of a transaction is entirely the addressed block's, as documented in that block's book. Software that polls a slow register (a UART line-status register after a transmit, for example) sees that block's documented latency, unchanged by the subsystem layer.

A consequence worth knowing for debug: while PREADY is low, the address and control must be held stable per the APB spec — the crossbar holds the *decode* stable with them, so a stalled transaction cannot migrate to a different window mid-wait.

12.5 Write Strobes and Protection

PSTRB and PPROT are forwarded to the selected block on every transaction. What the block does with them is the block's own documented behaviour:

- PSTRB — byte lane enables for writes. Blocks that implement byte-granular register writes honour it; blocks with word-only registers document that in their own book. Masters should not assume more than the target block promises.
- PPROT — protection attributes. The RLB blocks accept and ignore it (no protection checking at this level), as each per-block book states. It exists on the boundary because APB4 requires the signals to be carried; a system-level protection scheme would be enforced above `rlb_top`.

12.6 Error Responses

A transaction completes with PSLVERR asserted for exactly two reasons:

1. **Decode miss.** The address matches no window. The generated crossbar's decode-miss path completes the transaction with PSLVERR rather than holding PREADY low — an unmapped access can never hang the bus. Read data on a miss is undefined; software must treat the transaction as failed. This path is a property of the generator and is the reason the crossbar is regenerated rather than hand-edited.

2. **Block error.** The addressed block asserts its own PSLVERR (for example, an access to a register offset that block does not implement). The block's book owns the specifics; the boundary simply returns whatever the block raises.

The two are distinguishable in hardware only by address: if the address is inside a decoded window, the error came from the block; if it is outside all ten windows, it came from the decode miss. The `func` suite proves the decode-miss case completes instead of hanging.

12.7 Rules an Integrator Must Honour

- Drive the **full system address**; the subsystem decodes PADDR[15:12] against BASE_ADDR internally and blocks see only PADDR[11:0].
- Never issue a new transaction to a different window while PREADY is low on the current one — standard APB, restated here because the crossbar's decode follows the held address.
- Tie PSTRB/PPROT per the APB4 rules if your master does not drive them.
- Expect errors to be *fast*, never *absent*: both failure modes complete with PSLVERR in bounded time.

12.8 Related Documents

- Top-Level Interface - the port list and the decode table this protocol serves
- The APB Crossbar Block - the generated fabric that implements this boundary
- Window Map - the ten windows the decode resolves to

13 RLB Top - Interrupt Pin Reference

13.1 Overview

This page answers the pin-level question: **which line does each block's interrupt land on, and where does each external interrupt pin go?** The routing intent — convention versus choice, OR-ed rather than substituted, IRQ2 never driven — is documented in The Interrupt Fabric. This page is the table a firmware or board engineer looks up a single pin in. Nothing here invents behaviour: every row is a signal on `rlb_top`'s port list, and the port list is in Top-Level Interface.

13.2 Block Interrupt Outputs, Pin by Pin

RTL pin	Fabric line	Master 8259	Slave 8259	IOAPIC pin	`rlb_irq_out`	Notes
`pit_timer_irq[0]`	IRQ0	IR0	—	0	yes	8254 channel-0 tick
`pit_timer_irq[2:1]`	none	—	—	—	yes	Channels 1-2 leave on their pins and the aggregate only
`hpet_legacy_irq0`	IRQ0	IR0	—	0	yes	Only while `HPET_CONFIG.legacy_replacement` is set; suppresses `hpet_timer_irq[0]` while set
`hpet_legacy_irq8`	IRQ8	—	IR0	8	yes	Only

RTL pin	Fabric line	Master 8259	Slave 8259	IOAPIC pin	rlb_irq_out	Notes
						while legacy replacement is set; suppresses hpet_timer_irq[1] while set
hpet_timer_irq[1:0]	none	—	—	—	yes	**Deliberately not routed** — no conventional legacy line; TIMER_INT_ROUTE_CAP reads 0 for the same reason
uart_irq	IRQ4	IR4	—	4	yes	COM1 by convention
rtc_alarm_irq	IRQ8	—	IR0	8	yes	Shares IRQ8 with rtc_second_irq and hpet_legacy_irq8
rtc_second_irq	IRQ8	—	IR0	8	yes	Periodic

RTL pin	Fabric line	Master 8259	Slave 8259	IOAPIC pin	rlb_irq_out	Notes
						interrupt
smb_interrupt	IRQ10	—	IR2	10	yes	**Choice** — parameter IRQ_SMBUS
pm_interrupt	IRQ9	—	IR1	9	yes	ACPI by convention; PM wake inputs (pm_rtc_alarm, pm_ext_wake_n, GPE banks) feed the PM block, not the fabric
gpio_irq	IRQ11	—	IR3	11	yes	**Choice** — parameter IRQ_GPIO
pic_int_out	none	—	—	—	yes	The master's delivered output. Fed back to nothing internally — a loop — it appears

RTL pin	Fabric line	Master 8259	Slave 8259	IOAPIC pin	`rlb_irq_out`	Notes
						only in the aggregate
`ioapic_irq_out_*`	n/a	—	—	—	no	A message interface (`valid`, `vector`, `dest`, `deliv_mode`, `dest_mode`), not a level line; consumed by the receiver above the subsystem

Rows saying “IR0” for the slave mean the slave’s interrupt input bit 0, which **is** IRQ8 in system numbering — the slave covers IRQ8-15. Rows saying “none” under Fabric reach the aggregate and their own pins only.

13.3 The Sixteen Legacy Lines As the Controllers See Them

IRQ	Master IR	Slave IR	IOAPIC pin	Source(s) on the fabric	Fixed or parameter
0	IR0	—	0	`pit_timer_irq[0]`, `hpet_legacy_irq0`	convention
1	IR1	—	1	none	—
2	—	—	—	**never driven** — cascade input,	—

IRQ	Master IR	Slave IR	IOAPIC pin	Source(s) on the fabric	Fixed or parameter
				forced from the slave's `INT`	
3	IR3	—	3	none	—
4	IR4	—	4	`uart_irq`	convention
5	IR5	—	5	none	—
6	IR6	—	6	none	—
7	IR7	—	7	none	—
8	—	IR0	8	`rtc_alarm_irq`, `rtc_second_irq`, `hpet_legacy_irq8`	convention
9	—	IR1	9	`pm_interrupt`	convention
10	—	IR2	10	`smb_interrupt`	parameter `IRQ_SMBUS`
11	—	IR3	11	`gpio_irq`	parameter `IRQ_GPIO`
12	—	IR4	12	none	—
13	—	IR5	13	none	—
14	—	IR6	14	none	—
15	—	IR7	15	none	—

IOAPIC pins 16-23 exist on the boundary (`IOAPIC_NUM_IRQS` = 24) and are sourced only by the board, through `ioapic_irq_in` — the subsystem drives no fabric line above 15.

13.4 External Interrupt Input Pins

Pin	Feeds	Notes
`pic_irq_in[7:0]`	Master, OR-ed with boot-interrupt reroute and fabric lines 0-7	Bit 2 is **ignored**: masked off before the slave's `INT` is forced onto IR2, so driving it reaches nothing

Pin	Feeds	Notes
`pic_irq_in[15:8]`	Slave, OR-ed with fabric lines 8-15	System IRQ8-15
`ioapic_irq_in[23:0]`	IOAPIC pins, OR-ed with the zero-extended fabric; also feeds `u_ioapic_boot_intx`	Pins 0-7 can be rerouted onto legacy inputs when the IOAPIC is not delivering them (see below); a reroute onto pin 2 is a dead entry
`ioapic_eoi_in`, `ioapic_eoi_vector[7:0]`	IOAPIC EOI input	End-of-interrupt from the local APIC side
`ioapic_irq_out_ready`, `ioapic_irq_out_retry`	Delivery handshake inputs	Tie `retry` low if the receiver always accepts

13.5 The Aggregate

`rlb_irq_out` is a pure OR of: `hpet_timer_irq`, `hpet_legacy_irq0`, `hpet_legacy_irq8`, `pic_int_out`, `pit_timer_irq`, `rtc_alarm_irq`, `rtc_second_irq`, `smb_interrupt`, `pm_interrupt`, `gpio_irq`, `uart_irq`. It exists for a SoC that wants one interrupt input rather than ten pins; it drives nothing inside the subsystem, which is why including `pic_int_out` in it is safe. The IOAPIC's message-style delivery output is **not** part of it.

13.6 Scope Boundaries Restated

Three integration decisions live at this boundary and are choices, not defects:

- `hpet_timer_irq` is not routed into the fabric (first table, third row).
- The two MSI configuration outputs (`cfg_msi_addr`, `cfg_msi_data`) are connected and left open — `ioapic_msi_emit` is not instantiated.
- Boot-interrupt rerouting exists (`u_ioapic_boot_intx`) but its pin-to-legacy map defaults to the identity mapping for pins 0-7 and no reroute above that; the pin-2 entry is dead because IRQ2 is the cascade input.

13.7 Related Documents

- The Interrupt Fabric - why the table above looks the way it does
- The Cascade Cross-Connect - why IRQ2 is never driven
- Initialization - programming the controllers to receive these lines

- Use Cases - one interrupt routed end to end

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

14 RLB Top - Initialization and Programming Guide

14.1 Scope

This chapter is the **subsystem** bring-up: the order in which the pieces have to be configured, and the parts that only exist at the integration level - the interrupt controllers, the IOAPIC redirection table, and the cascade. Programming a block's own function (setting a timer period, configuring a UART's baud rate) belongs to that block's specification.

The sequences below are the ones the integration test suite executes, which is why they are known to work rather than merely plausible.

There is also an executable transcription of this chapter: `dv/host/rlb_bringup_programs.py` runs steps 1-4 against any `write32`/`read32` bus (a board port binds the same program to a UART bridge), and `dv/tests/test_rlb_top_bringup.py` proves the book sufficient by driving bring-up through that program alone — it never calls the DV helpers this chapter was transcribed from. Edit this chapter, then run that test: it caught two wrong register offsets in this chapter's first edition (`IOWIN`, `PIC_STATUS`), which is exactly the drift it exists to catch.

14.2 Address Helper

Every register access in this chapter goes through a window. One helper keeps the rest readable:

```
#define RLB_BASE      0xFEC00000u
#define RLB_WINDOW    0x1000u

#define WIN_HPET      0
#define WIN_PIC       1      /* master 8259 */
#define WIN_PIT       2
#define WIN_RTC       3
#define WIN_SMBUS     4
#define WIN_PM        5
#define WIN_IOAPIC    6
#define WIN_GPIO      7
#define WIN_UART      8
#define WIN_PIC_SLAVE 9      /* slave 8259 */

static inline volatile uint32_t *rlb_reg(unsigned window, unsigned
offset)
{
    return (volatile uint32_t *)(RLB_BASE + window * RLB_WINDOW +
offset);
}
```

14.3 Bring-Up Order

Step	What	Why here
1	Release reset, confirm the subsystem answers	Nothing below is meaningful if decode is broken
2	Decide the interrupt topology	Determines whether step 3 is the single or cascade sequence
3	Configure the 8259 (single **or** cascaded pair)	The controllers must be initialised before any source is enabled
4	Configure IOAPIC redirection entries	Entries reset **masked**; an unconfigured entry silently delivers nothing
5	Enable the peripheral blocks	Only now can a source assert without being lost
6	Verify	Read back status rather than assuming

The ordering constraint that actually matters is steps 3 and 4 before step 5. A block enabled before its controller is configured asserts into a controller that is still in its initialisation state, and in edge-triggered mode the result is a latched interrupt nobody is ready to service.

14.4 Step 1: Confirm the Subsystem Answers

Each window has a read-safe probe register. Reading all ten is the cheapest possible confirmation that the crossbar and every block are alive.

```
/* Read-safe probe per window. */
static const struct { unsigned window, offset; const char *name; }
probes[] = {
    { WIN_HPET,      0x000, "hpet"      },  /* HPET_ID, read-only
identity */
    { WIN_PIC,       0x000, "pic"       },  /* PIC_CONFIG */
    { WIN_PIT,       0x000, "pit"       },  /* PIT_CONFIG */
    { WIN_RTC,       0x000, "rtc"       },  /* RTC_CONFIG */
    { WIN_SMBUS,     0x000, "smbus"     },  /* SMBUS_CONTROL */
    { WIN_PM,        0x000, "pm"        },  /* ACPI_CONTROL */
    { WIN_IOAPIC,    0x000, "ioapic"    },  /* IOREGSEL */
    { WIN_GPIO,      0x000, "gpio"      },  /* GPIO_CONTROL */
    { WIN_UART,      0x020, "uart"      },  /* UART_SCR -- see the
```

```
warning */
    { WIN_PIC_SLAVE, 0x000, "pic_slave" },  /* PIC_CONFIG on the slave
*/
};
```

Do not probe the UART at offset `0x000`. That is the receive buffer, and reading it pops a byte off the RX FIFO. Use the scratch register at `0x020`. This is a real trap: a "harmless" identity read that destroys received data.

An unmapped address is also a useful probe in the other direction: it must complete with `PSLVERR` asserted rather than hang. If it hangs, the crossbar is not the generated one.

14.5 Step 2: Choose the Interrupt Topology

Topology	Use when	Configure
Single 8259	Nothing you care about is on IRQ8-15	Master only, `SNGL` set
Cascaded pair	Anything on IRQ8-15 - which includes the RTC, ACPI, SMBus and GPIO	Both controllers, `SNGL` clear
IOAPIC	You want vectored delivery with per-pin destination control	Redirection entries, step 4

The fabric feeds all of these simultaneously. Choosing one does not disable the others.

Note which sources land where: **IRQ0 and IRQ4 reach the master; IRQ8, IRQ9, IRQ10 and IRQ11 reach the slave.** With four of the six sourcing blocks on the slave controller, the single-controller topology is rarely what you want.

14.6 Step 3a: Single-Controller Configuration

Master only, in single mode. This is enough to get `INT` asserting.

```
#define PIC_CONFIG 0x000
#define PIC_ICW1   0x004
#define PIC_ICW2   0x008
#define PIC_ICW3   0x00C
#define PIC_ICW4   0x010
#define PIC_OCW1   0x014
#define PIC_STATUS 0x028

int pic_init_single(uint8_t vector_base)   /* vector_base e.g. 0x20 */
{
    *rlb_reg(WIN_PIC, PIC_CONFIG) = 0x1;          /* enable, NOT
```

```
init_mode */
    *rlb_reg(WIN_PIC, PIC_ICW1)   = 0x10 | 0x02 | 0x01;  /* marker |
SNGL | IC4 */
    *rlb_reg(WIN_PIC, PIC_ICW2)   = vector_base;
    /* No ICW3: SNGL is set, so the controller does not expect one. */
    *rlb_reg(WIN_PIC, PIC_ICW4)   = 0x01;         /* 8086 mode */
    *rlb_reg(WIN_PIC, PIC_OCW1)   = 0x00;         /* unmask all */

    return (*rlb_reg(WIN_PIC, PIC_STATUS) & 1) != 0;   /* init
complete */
}
```

`PIC_CONFIG` must set the enable bit without `init_mode`. Setting `init_mode` here sends the initialisation state machine back to its idle state once the sequence completes, and the controller ends up unconfigured. This is the single most likely way to get a controller that accepts every write and then does nothing.

`ICW1` here is `0x13`: the marker bit, `SNGL`, and `IC4`. Level-triggered mode (`LTIM`) is left clear, so the controller is edge-triggered - which matters when clearing an interrupt, because `INT` latches until acknowledged.

14.7 Step 3b: Cascaded-Pair Configuration

Both controllers, as a master and slave pair. Anything arriving on IRQ8-15 requires this.

```
int pic_init_cascade(uint8_t master_base, uint8_t slave_base)  /* e.g.
0x20, 0x28 */
{
    /* ---- MASTER, window 1 */
    *rlb_reg(WIN_PIC, PIC_CONFIG) = 0x1;
    *rlb_reg(WIN_PIC, PIC_ICW1)   = 0x10 | 0x01;  /* marker | IC4,
SNGL CLEAR */
    *rlb_reg(WIN_PIC, PIC_ICW2)   = master_base;
    *rlb_reg(WIN_PIC, PIC_ICW3)   = 0x04;         /* a slave is
attached on IR2 */
    *rlb_reg(WIN_PIC, PIC_ICW4)   = 0x01;
    *rlb_reg(WIN_PIC, PIC_OCW1)   = 0x00;         /* unmask all, IR2
included */

    /* ---- SLAVE, window 9 */
    *rlb_reg(WIN_PIC_SLAVE, PIC_CONFIG) = 0x1;
    *rlb_reg(WIN_PIC_SLAVE, PIC_ICW1)   = 0x10 | 0x01;
    *rlb_reg(WIN_PIC_SLAVE, PIC_ICW2)   = slave_base;
    *rlb_reg(WIN_PIC_SLAVE, PIC_ICW3)   = 0x02;   /* I hang off master
IR2 */
    *rlb_reg(WIN_PIC_SLAVE, PIC_ICW4)   = 0x01;
    *rlb_reg(WIN_PIC_SLAVE, PIC_OCW1)   = 0x00;
```

```
    return (*rlb_reg(WIN_PIC,       PIC_STATUS) & 1)
        && (*rlb_reg(WIN_PIC_SLAVE, PIC_STATUS) & 1);
}
```

Four things must all be true for a slave interrupt to reach the CPU, and omitting any one of them looks exactly like a broken interrupt fabric:

1. The master is in cascade mode - `SNGL` **clear** in `ICW1`
2. The master's `ICW3` marks IR2 as carrying a slave (`0x04`)
3. The slave's `ICW3` states which master line it hangs off (`0x02`)
4. Master IR2 is unmasked in `OCW1`

With `SNGL` clear, `ICW3` is mandatory. The initialisation state machine waits for it. Skipping it leaves the controller stuck mid-sequence.

Note that `master_base` and `slave_base` differ by 8 (`0x20` and `0x28`): the slave's eight lines get their own vector block.

14.8 Step 4: IOAPIC Redirection Entries

The IOAPIC is programmed indirectly: write the register index to `IOREGSEL`, then the value to `IOWIN`.

```
#define IOAPIC_IOREGSEL  0x000
#define IOAPIC_IOWIN     0x004
#define IOAPIC_BOOTINTX  /* see ioapic_mas ch05 for the offset */

static void ioapic_write(unsigned selector, uint32_t value)
{
    *rlb_reg(WIN_IOAPIC, IOAPIC_IOREGSEL) = selector;
    *rlb_reg(WIN_IOAPIC, IOAPIC_IOWIN)    = value;
}

/* Redirection entries are two 32-bit halves per pin. */
static unsigned rte_lo(unsigned irq) { return 0x10 + irq * 2; }
static unsigned rte_hi(unsigned irq) { return 0x11 + irq * 2; }

void ioapic_arm(unsigned irq, uint8_t vector, int masked)
{
    ioapic_write(rte_lo(irq), vector | ((masked ? 1u : 0u) << 16));
    ioapic_write(rte_hi(irq), 0);      /* destination */
}
```

Redirection entries reset masked. This is the step most often missed, and its failure mode is the worst kind: the source reaches the IOAPIC's pin, the pin is correct, and no message is ever delivered. That is indistinguishable from a broken route unless you know to check the mask bit.

On vectors: the assignment is yours. The published typical-assignment table suggests `0x20` through `0x2F` for the legacy lines. The integration test suite uses `0x40 + irq`, which is a **test convention, not a hardware property** - convenient because the vector then says which line produced it. Nothing in the RTL constrains the choice.

14.9 Step 5: Enable the Blocks

Per-block, from that block's specification. The only integration-level rule is that this comes last.

If you enable the HPET's legacy-replacement mode, be aware it takes over IRQ0 and IRQ8: HPET timers 0 and 1 then drive `hpet_legacy_irq0` and `hpet_legacy_irq8` into those lines and are suppressed on `hpet_timer_irq`. You should silence the 8254 channel 0 and the RTC periodic interrupt to match, or two sources will be competing for one line - legally, since the fabric ORs them, but confusingly.

14.10 Step 6: Verify

Check	How
Decode is sound	All ten probes read without error; an unmapped address sets `PSLVERR`
Controllers initialised	`PIC_STATUS` bit 0 set on each configured controller
Cascade consistent	Master IR2 follows the slave's `INT`. It should rise for an IRQ8-15 source - that is the cascade working, not a fault
A route works	Assert a source, confirm `pic_int_out`, and confirm an IOAPIC delivery with the vector you programmed
Anything at all is pending	`rlb_irq_out` is asserted while any block interrupt is

14.11 Clearing an Interrupt

In edge-triggered mode `pic_int_out` latches until the controller is acknowledged. Software should acknowledge and issue EOI normally.

The integration test suite instead resets the whole subsystem between phases, because the acknowledge and EOI paths carry their own block-level limitations that would make an

integration test fail on somebody else's known defect. That is a verification convenience, not guidance for firmware. It is mentioned here only because it explains a real consequence: **a reset drops all controller and IOAPIC configuration**, so anything that resets must redo steps 3 and 4.

14.12 Related Documents

- Use Cases - complete routing paths, end to end
- Architecture - what the fabric does with a source
- Window Map - where each block's registers are specified
- pic_8259_mas, ioapic_mas - the controllers' own specifications

15 RLB Top - Use Cases

Six worked paths through the subsystem. Each one is exercised by the integration test suite, so each is known to work as described.

15.1 Use Case 1: A UART Interrupt Through the Master Controller

The simplest routing path: a source on a master line.

Stage	Signal
Source	`uart_irq` from `u_uart`
Fabric line	IRQ4 (`IRQ_UART`, convention: COM1)
Master input	`w_master_pic_irq[4]`
Output	`pic_int_out`

```
pic_init_cascade(0x20, 0x28);       /* or pic_init_single(0x20) --
IRQ4 is a master line */
uart_enable_interrupts();           /* see uart_16550_mas */
/* uart_irq -> w_fabric_irq[4] -> master IR4 -> pic_int_out */
```

The vector the CPU sees is `master_base + 4`, so `0x24` with a base of `0x20`.

15.2 Use Case 2: A GPIO Interrupt Through the Cascade

A source on a slave line. This is the common case, not the exception - four of the six sourcing blocks land on the slave controller.

Stage	Signal
Source	`gpio_irq` from `u_gpio`
Fabric line	IRQ11 (`IRQ_GPIO`, a **choice**, parameterised)
Slave input	slave IR3, because 11 - 8 = 3
Slave output	`w_spic_int`
Master input	`w_master_pic_irq[2]`, forced from the slave
Output	`pic_int_out`, returning the **slave's** vector

```
pic_init_cascade(0x20, 0x28);       /* mandatory: IRQ11 is a slave
line */
gpio_enable_interrupt(pin);         /* see gpio_mas */
```

The vector is `slave_base + 3`, so `0x2B` with a slave base of `0x28`. The master returns the slave's vector in place of its own - that is what the cascade acknowledge path is for.

Master IR2 rising here is **correct behaviour**, not a spurious cascade interrupt. The invariant to check is that IR2 equals the slave's `INT`, never that IR2 stays low.

15.3 Use Case 3: The Same Source Through the IOAPIC

The fabric feeds the IOAPIC in parallel with the controllers. No source change is needed - only IOAPIC configuration.

```
ioapic_arm(11, 0x40 + 11, /* masked = */ 0);   /* unmask IRQ11's entry
*/
gpio_enable_interrupt(pin);
/* Delivery appears on ioapic_irq_out_valid with vector 0x4B. */
```

Both paths are live at once: the same GPIO assertion reaches the cascade *and* produces an IOAPIC delivery. If you want only one, mask the other - mask the IOAPIC entry, or mask the line in the controller's `OCW1`.

Remember the mask default. An entry you never programmed is masked, and a masked entry delivers nothing while looking perfectly correct from the source side.

15.4 Use Case 4: HPET Legacy Replacement

Replacing the 8254 tick and the RTC periodic interrupt with HPET timers.

HPET output	Line	Replaces
`hpet_legacy_irq0`	IRQ0	8254 channel 0 tick
`hpet_legacy_irq8`	IRQ8	RTC periodic interrupt

```
hpet_set_legacy_replacement(1);     /* see hpet_mas */
pit_disable_counter(0);             /* avoid two sources on IRQ0 */
rtc_disable_periodic();             /* avoid two sources on IRQ8 */
```

The HPET core suppresses timers 0 and 1 on `hpet_timer_irq` while this mode is active, so a single event cannot arrive twice from the HPET itself. Silencing the 8254 and the RTC is still your job: the fabric ORs all three onto those lines, so leaving them enabled is legal and produces interrupts from sources you thought you had replaced.

The general HPET timers are unaffected, and remain unrouted - they reach `rlb_irq_out` and their own pins only.

15.5 Use Case 5: Boot-Interrupt Rerouting

Catching a source whose IOAPIC entry is masked, so it is not lost during boot before the IOAPIC is configured.

```
ioapic_arm(irq, vector, /* masked = */ 1);      /* IOAPIC will NOT
deliver */
ioapic_write(IOAPIC_BOOTINTX, 1);               /* reroute masked pins
to the PIC */
/* The source now reaches the legacy controller instead. */
```

The map is the identity mapping for IOAPIC pins 0-7 onto legacy inputs 0-7, and no reroute above pin 7.

Pin 2 cannot be rerouted. The map still names legacy input 2, but master IR2 is masked off every external source and driven from the slave controller, so a reroute onto pin 2 reaches nothing. If you need pin 2's source on the legacy path, pick a different legacy input.

Rerouting is disabled at reset, so the master's interrupt input is exactly `pic_irq_in` until you perform the second write above.

15.6 Use Case 6: One Interrupt Line for the Whole Subsystem

For a system that would rather not model ten interrupt sources.

```
/* Wire rlb_irq_out to your controller. On assertion, poll to find the
source. */
if (rlb_irq_out_asserted()) {
    /* Check each block's status register to identify the source. */
}
```

`rlb_irq_out` asserts while **any** block interrupt is asserted, including `pic_int_out` itself. It carries no vector and no priority - it is a pure OR, and identifying the source means polling. It drives nothing inside the subsystem, which is precisely why including the controller's own output in it is safe.

This is also the path that sees the general HPET timers, which no other path does.

15.7 Coincident Sources

The fabric is combinational and per-line, so simultaneous sources are handled by construction: each lands on its own line, and lines that share a source are OR-ed. The integration suite exercises two, three and four coincident sources, spanning both controllers - for example the UART on master IR4 while SMBus, ACPI and GPIO assert together under the cascade.

Two practical notes from that testing:

- **Some sources are slow to assert.** ACPI takes tens of clocks from `pm_gpe_events` to `pm_interrupt`, and SMBus considerably longer. Code that expects a source to be asserted a few cycles after its stimulus will sample too early.
- **Priority is the controller's job, not the fabric's.** The fabric presents every asserted line at once; which one the CPU services is decided by the 8259 priority logic, or by the IOAPIC's delivery configuration.

15.8 Related Documents

- Initialization - the bring-up sequences these use cases assume
- Architecture - the fabric, cascade and aggregate
- Window Map

16 RLB Top - Power Management: Sleep and Wake Across Blocks

16.1 Overview

Sleep in this subsystem belongs to exactly one block: the PM/ACPI controller on window 5. The other nine blocks do not implement power states, are not clock-gated by the PM block inside `rlb_top`, and keep running on `pclk` through a sleep. What the PM block *does* produce on its output pins — `pm_clock_gate_en[31:0]` and `pm_power_domain_en[7:0]` — is a board-level clock-and-power policy that a consuming SoC may honour, ignore, or wire to nothing. This page sequences the part software controls: arming wake sources, entering S1/S3/S5, and reading the subsystem back to S0. Register-level field detail is the PM/ACPI book's; this page is the order of operations and the subsystem-level facts that change it.

All addresses below are in window 5 (base `RLB_BASE + 0x5000`); see Chapter 5 and the pm_acpi register chapter for the field tables.

16.2 Who Participates

Block	Role in sleep
PM/ACPI	Owns it. The sleep FSM, wake latches, and the gate/domain outputs all live here
8259 pair, IOAPIC	Unchanged. Their programming survives sleep — an interrupt controller does not need re-initialization after wake
HPET, PIT, RTC, GPIO, UART, SMBus	Ignore sleep inside the subsystem. Their `*_clk`/`*_resetn` pins are unused (`CDC_ENABLE(0)`) and nothing gates their `pclk`. If a system wants them quiet in S3, the board consumes the PM gate outputs above the subsystem
APB fabric	Stays live. The crossbar decodes regardless of power state, which is what lets software read `WAKE_STATUS` after a wake

Two consequences follow. First, there is no per-block quiesce sequence inside `rlb_top` — software does not park the UART or silence the PIT before sleeping; it simply stops caring, because

nothing in the subsystem asserts their interrupts into the fabric unless their own event occurs. Second, `pm_clock_gate_en` / `pm_power_domain_en` are *outputs for a board*, not a mechanism this subsystem enforces: in simulation they can be observed and ignored.

16.3 Step 1: Bring the PM Block Up

The PM block is one of the ten windows brought up by Initialization step 5 — enable blocks — with one extra gate before it matters: `ACPI_CONTROL.acpi_enable` (0x000 bit 0) is the SCI_EN-like master switch. While it is 0, `pm_interrupt` is held low and no GPE event is captured, though `PM1_STATUS` and `WAKE_STATUS` still record their events. Set it before arming anything that must interrupt.

16.4 Step 2: Arm the Wake Sources

A source wakes the machine only when both halves are programmed:

1. **Global wake mask** — `WAKE_ENABLE` (0x064): bits for GPE, power button, RTC alarm, and external wake.
2. **Per-bank enables** — for GPE sources, program `GPE0_ENABLE` / `GPE1_ENABLE` (0x038-0x03C, 0x09C-0x0A0) and the matching `GPE0_WAKE_EN` / `GPE1_WAKE_EN` registers, and choose edge or level per source with `GPE0_TRIGGER` / `GPE1_TRIGGER`. The PM/ACPI book's GPE section owns the interrupt/wake split; the short version is that a source can be armed to wake a sleeping machine without interrupting a running one.

The non-GPE wake inputs are pins, and three of them need board attention, stated here because they are the usual "sleep works but never wakes" causes:

Wake input	Pin	Board responsibility
Power button	`pm_power_button_n`	Drive from the chassis switch; debounce window in `BUTTON_TIMING` (0x070)
Sleep button	`pm_sleep_button_n`	Optional; mask with `PM1_ENABLE.slpbtn_en` if absent
RTC alarm	`pm_rtc_alarm`	**Not wired to the RTC inside `rlb_top`** — `rtc_alarm_irq` is an output pin and `pm_rtc_alarm` is an input

Wake input	Pin	Board responsibility
		pin. Connect them at the board/SoC level if the RTC should wake the system
External wake	`pm_ext_wake_n`	Tie inactive if unused

16.5 Step 3: Enter Sleep

Program `PM1_CONTROL` (0x010):

1. Write `sleep_type` (bits 2:0): 1 = S1, 3 = S3, 5 = S5. Other values are treated as 0 and the FSM stays in S0.
2. Write `sleep_enable` (bit 3) = 1. This is a **one-shot**: the bit self-clears, and the core rising-edge-detects it, so one write is one sleep request and a wake that returns the machine to S0 is not undone by the still-programmed `sleep_type`.

On a board with real power rails, enable the rail sequencer first (`PWR_SEQ_CONFIG`, 0x07C): `seq_enable` turns the clock/rail moves into an ordered walk — clocks gated first on the way down, rails 7 to 0; rails 0 to 7 first on the way up, clocks ungated last — with optional per-rail acknowledge and a programmable inter-step delay. Without it the transitions are instant, which is correct in simulation and wrong on a board. A rail that never acknowledges stalls the walk deliberately; `PWR_SEQ_STATUS` (0x080) is where to look (`seq_busy` parked on a rail index). There is no timeout, by design.

The hardware then walks the FSM through TRANSITION into the programmed state. What the state means at the outputs:

State	`pm_clock_gate_en`	`pm_power_domain_en`	Leaving it
S1	`CLOCK_GATE_CTRL & 0x3` — only blocks 0-1 may stay clocked	`POWER_DOMAIN_CTRL` unchanged	Any armed wake source
S3	0 — everything gated	`POWER_DOMAIN_CTRL & 0x1` — domain 0 always on	Any armed wake source
S5	0	`POWER_DOMAIN_CTRL & 0x1`	Pulses `pm_sys_reset_req`

State	pm_clock_gate_en	pm_power_domain_en	Leaving it
			: a wake from S5 is a **boot**, not a resume

`ACPI_CONTROL.current_state` (bits 5:4) reports the encoding, not the ACPI number: 0 = S0, 1 = S1, 2 = S5, 3 = S3. It reads 0 while the FSM is in TRANSITION.

16.6 Step 4: Wake

An armed source fires while the FSM is in S1 or S3 and the core latches the wake request — the latch outranks the still-programmed `sleep_type` while the FSM is in TRANSITION, which is why a button press landing during entry does not bounce the machine back into sleep. The FSM returns through TRANSITION to S0, the latch drops, and the walk (if enabled) re-powers rails in reverse order.

Software then:

1. Reads `WAKE_STATUS` (0x060) to identify the source — `gpe_wake`, `pwrbtn_wake`, `rtc_wake`, `ext_wake` — and writes 1s to clear it.
2. Clears the matching status that fed the interrupt: `ACPI_STATUS` (0x004), and `PM1_STATUS` (0x014) for the button/RTC paths, and the GPE status registers for a GPE wake. `pm_interrupt` is a level — the OR of the enabled sticky bits — and deasserts only when every enabled source has been cleared.
3. If the sleep was S5, treats the wake as a boot: `RESET_STATUS` reads differently, and whatever the system wired to `pm_sys_reset_req` has reset the machine.

One corner the PM book records: a wake event in the exact cycle of the sleep request is **not** latched as a wake. The sticky status bits and the level interrupt still record it, the machine enters the programmed state, and the next wake event brings it back. Software that must not sleep through such an event checks the status registers after entry.

16.7 Step 5: The Interrupt Path for ACPI Events

`pm_interrupt` leaves the PM block on the fabric's IRQ9 line — ACPI by convention — and therefore reaches the slave 8259 (IR1) and IOAPIC pin 9 like any other routed source (see Interrupt Pin Reference). The SCI handler is just a legacy interrupt handler: program the controllers during initialization — step 3 for the 8259s, step 4 for the IOAPIC — to deliver IRQ9, and the PM block needs no special routing. GPE events additionally depend on `ACPI_INT_ENABLE.gpe_int_enable` and the GPE per-source enables before they contribute to the pin.

16.8 What Sleep Does Not Disturb

- **Interrupt controller programming.** The 8259 pair and IOAPIC hold their configuration across sleep; there is no re-init in the wake path.
- **The other blocks' state.** UART FIFOs, PIT counters, RTC time, GPIO configuration — all of it survives, because nothing inside the subsystem gates them. Only a board that consumes the PM gate outputs can change that, and then it is the board's contract, not this book's.
- **The address map.** Every window answers the same way after wake as before; the subsystem performs no remap on sleep or wake.

16.9 Related Documents

- Initialization - the bring-up these steps assume
- Use Cases - routing an interrupt end to end (the IRQ9 path)
- The Interrupt Fabric - how IRQ9 reaches both controllers
- PM/ACPI register detail - `pm_acpi_mas` book, chapter 5

---

17 RLB Top - Address Map

17.1 rlb_top Has No Registers

`rlb_top` implements no registers of its own. Every address it answers belongs to a block it instantiates. This chapter is therefore the **window map** - how an address is decoded, and which specification documents the registers inside each window.

17.2 Decode

Property	Value
Base	`BASE_ADDR`, default `0xFEC0_0000`
Window size	`0x1000` (4KB)
Windows	10, indices 0-9
Slave index	`PADDR[15:12]`
Offset within a block	`PADDR[11:0]`
Reserved	none - all ten windows are assigned
Unmapped	completes with `PSLVERR`; does not hang

The decode is a clean power of two: the window number *is* `PADDR[15:12]`, and the address a block sees is the low twelve bits. A block's own register map is therefore documented in offsets from `0x000`, and the subsystem address is `BASE_ADDR + window * 0x1000 + offset`.

17.3 Window Map

Window	Address range	Block	Register map
0	`0xFEC0_0000` - `0xFEC0_0FFF`	HPET	hpet_mas ch05_registers/01_register_map.md
1	`0xFEC0_1000` - `0xFEC0_1FFF`	8259 PIC (master)	pic_8259_mas ch05_registers/01_register_map.md
2	`0xFEC0_2000` - `0xFEC0_2FFF`	8254 PIT	pit_8254_mas ch05_registers/01_register_map.md
3	`0xFEC0_3000` -	RTC	rtc_mas

Window	Address range	Block	Register map
	0xFEC0_3FFF		ch05_registers/01_register_map.md
4	0xFEC0_4000 - 0xFEC0_4FFF	SMBus	smbus_mas ch05_registers/01_register_map.md
5	0xFEC0_5000 - 0xFEC0_5FFF	PM/ACPI	pm_acpi_mas ch05_registers/01_register_map.md
6	0xFEC0_6000 - 0xFEC0_6FFF	IOAPIC	ioapic_mas ch05_registers/01_register_map.md
7	0xFEC0_7000 - 0xFEC0_7FFF	GPIO	gpio_mas ch05_registers/01_register_map.md
8	0xFEC0_8000 - 0xFEC0_8FFF	UART 16550	uart_16550_mas ch05_registers/01_register_map.md
9	0xFEC0_9000 - 0xFEC0_9FFF	8259 PIC (cascade slave)	pic_8259_mas ch05_registers/01_register_map.md

Windows 1 and 9 are the same block with the same register map, wired as a master and slave pair. The registers are identical; what differs is the cascade configuration written into them. See Initialization.

17.4 Read-Safe Probe Registers

For confirming a window is alive without disturbing it.

Window	Block	Probe offset	Register
0	HPET	0x000	HPET_ID, read-only identity
1	PIC (master)	0x000	PIC_CONFIG
2	PIT	0x000	PIT_CONFIG
3	RTC	0x000	RTC_CONFIG
4	SMBus	0x000	SMBUS_CONTROL

Window	Block	Probe offset	Register
5	PM/ACPI	`0x000`	`ACPI_CONTROL`
6	IOAPIC	`0x000`	`IOREGSEL`
7	GPIO	`0x000`	`GPIO_CONTROL`
8	UART 16550	**`0x020`**	`UART_SCR` scratch - **not** `0x000`
9	PIC (slave)	`0x000`	`PIC_CONFIG`

The UART is the exception and it matters. Offset `0x000` in the UART window is the receive buffer; reading it pops a byte off the RX FIFO. Probe the scratch register at `0x020` instead. An identity read that quietly consumes received data is a genuinely difficult bug to find.

17.5 The IOAPIC Is Indirect

The IOAPIC occupies a 4KB window but exposes only two directly addressed registers. Everything else is reached by writing an index to `IOREGSEL` and then the data to `IOWIN`. The redirection table in particular is two 32-bit halves per pin, reached through that indirection - so an IOAPIC “register offset” in the IOAPIC specification is a selector value, not a window offset.

17.6 Interrupt Line Assignments

Not registers, but the other half of the programming surface. The line numbers are fixed in hardware for convention assignments and parameterised for choices.

Line	Source	Basis	Controller input
IRQ0	`pit_timer_irq[0]` OR `hpet_legacy_irq0`	convention	master IR0
IRQ2	cascade only - never driven by the fabric	convention	master IR2, from the slave’s `INT`
IRQ4	`uart_irq`	convention	master IR4
IRQ8	`rtc_alarm_irq` OR `rtc_second_irq` OR `hpet_legacy_irq8`	convention	slave IR0
IRQ9	`pm_interrupt`	convention	slave IR1
IRQ10	`smb_interrupt`	**choice**, parameter `IRQ_SMBUS`	slave IR2

Line	Source	Basis	Controller input
IRQ11	`gpio_irq`	**choice**, parameter `IRQ_GPIO`	slave IR3

Each line also reaches the IOAPIC pin of the same number. The IOAPIC's vector for a pin is whatever software programs into that pin's redirection entry; there is no hardware-fixed vector.

17.7 Related Documents

- Top-Level Interface - the decode contract and port list
- Initialization - the address helper and bring-up order
- Architecture - how a line reaches its destinations

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

18 Retro Legacy Blocks - Product Requirements Document

Component: Retro Legacy Blocks (RLB) - Production-Quality Legacy Peripherals **Version:** 1.0
Status: Active Development - HPET Production Ready **Last Updated:** 2025-10-29

18.1 1. Overview

18.1.1 1.1 Purpose

The Retro Legacy Blocks (RLB) component provides production-quality implementations of legacy peripheral blocks based on proven peripheral designs. These blocks are designed to be reusable, well-tested, and suitable for both FPGA and ASIC implementation.

18.1.2 1.2 Design Philosophy

"Retro" - Proven Architectures: - Implements time-tested peripheral designs from successful platforms - Focuses on simplicity, reliability, and well-understood behavior - Prioritizes production-readiness over experimental features

"Legacy" - Time-Tested Interfaces: - Based on proven peripheral interface specifications - Suitable for systems requiring retro-compatible peripheral compatibility - APB-based interface for easy integration

"Blocks" - Modular Collection: - Each peripheral is independent and self-contained - Clear separation between different blocks (rtl/hpet/, rtl/gpio/, etc.) - Can be used individually or wrapped into integrated subsystem

18.1.3 1.3 Target Applications

- Retro-compatible platform compatibility layers
- Embedded systems requiring legacy peripheral interfaces
- FPGA-based system emulation
- Educational platforms demonstrating classic peripheral designs
- Mixed-vintage SoC integration (modern + legacy interfaces)

18.2 2. Implemented Blocks

18.2.1 2.1 HPET - High Precision Event Timer

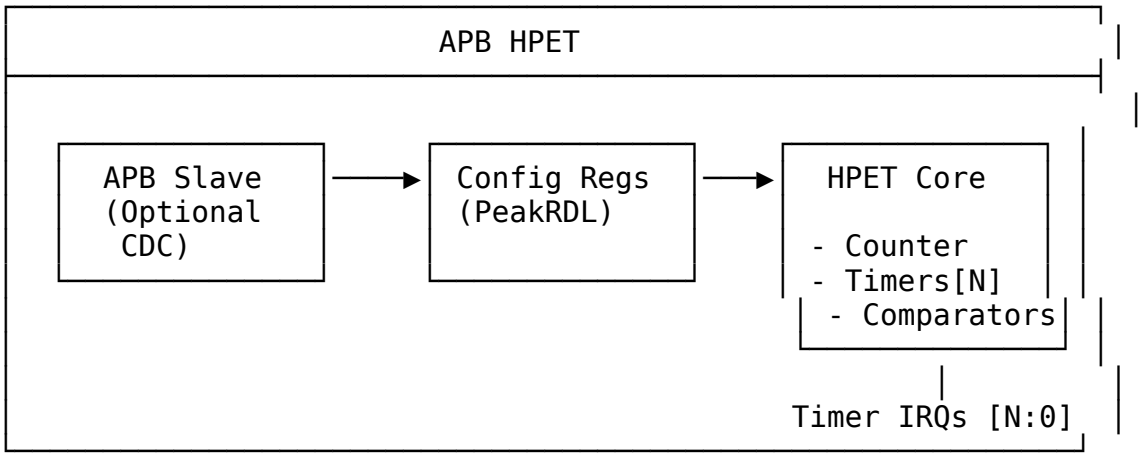
Status: Production Ready (5/6 configurations 100% passing) **RTL Location:** `rtl/hpet/` **Documentation:** `docs/hpet_mas/`

Key Features: - Configurable timer count: 2, 3, or 8 independent timers - 64-bit main counter for high-resolution timestamps - 64-bit comparators per timer - Operating modes: One-shot and periodic - Clock domain crossing: Optional CDC for timer/APB clock independence - APB4 interface: Standard AMBA APB protocol - PeakRDL integration: Register map generated from SystemRDL specification

Applications: - System tick generation - Real-time OS scheduling - Precise event timing - Performance profiling - Watchdog timers - Multi-rate timing domains

Test Coverage: - 6 configurations tested (2/3/8 timers, CDC on/off) - 5/6 configurations at 100% pass rate - 1 configuration at 92% (minor stress test timeout) - 12 test cases per configuration (gate/func/full)

Architecture:



Design Highlights: - Reset macro standardization (FPGA-friendly) - Per-timer data buses prevent corruption - Edge-triggered register write strobes (not level) - W1C status register for interrupt clearing - Optional asynchronous clock domains with handshake CDC

See: `docs/hpet_mas/hpet_mas_index.md` for complete HPET specification

18.3 3. Planned Blocks

18.3.1 3.1 8259 - Programmable Interrupt Controller (PIC)

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** High **Address:** `0xFEC0_1000 - 0xFEC0_1FFF` (4KB window, master of the cascaded pair)

Planned Features: - 8259A-compatible register interface - 8 interrupt request (IRQ) inputs - Cascadable (master/slave configuration) - Priority resolver (fixed and rotating priority) - Edge and level triggered modes - Interrupt mask register - End-of-Interrupt (EOI) handling - APB register interface

Applications: - Legacy interrupt management - PC-compatible systems - Hardware interrupt aggregation - Priority-based interrupt handling - Cascaded multi-level interrupt systems

18.3.2 3.2 8254 - Programmable Interval Timer (PIT)

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** High **Address:** `0xFEC0_2000 - 0xFEC0_2FFF` (4KB window)

Planned Features: - 8254-compatible register interface - 3 independent 16-bit counters - 6 programmable counter modes - Binary and BCD counting - Read-back command - Configurable clock input - Interrupt/output generation per counter - APB register interface

Counter Modes: - Mode 0: Interrupt on terminal count - Mode 1: Hardware retriggerable one-shot - Mode 2: Rate generator - Mode 3: Square wave mode - Mode 4: Software triggered strobe - Mode 5: Hardware triggered strobe

Applications: - System tick generation - Periodic timer interrupts - Square wave generation - Event counting - Legacy PC timer compatibility

18.3.3 3.3 GPIO - General Purpose I/O

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_7000 - 0xFEC0_7FFF` (4KB window)

Planned Features: - Configurable pin count (8, 16, 32 pins) - Per-pin direction control (input/output/bidirectional) - Input debouncing logic - Interrupt generation (rising/falling/both edges, level) - Output drive strength configuration - Pull-up/pull-down control - APB register interface

Applications: - LED control - Button inputs - Hardware control signals - Chip-select generation - Status monitoring

18.3.4 3.4 RTC - Real-Time Clock

Status: Implemented (apb4_rtc + MAS; see docs/rtc_mas – binary 24-hour timekeeping validated; BCD/12-hour modes and the production-mode time-set path are broken, tracked in issue #56) **Priority:** Medium **Effort:** 3-4 weeks **Address:** `0xFEC0_3000 - 0xFEC0_3FFF` (4KB window)

Planned Features: - 32.768 kHz clock input (typical RTC crystal frequency) - Seconds, minutes, hours, day, month, year tracking - Alarm functionality - Battery backup support (power domain considerations) - 24-hour or 12-hour (AM/PM) mode - Leap year handling - APB register interface

Applications: - System time-of-day tracking - Wake-on-alarm functionality - Timestamp generation - Power-aware applications

18.3.5 3.5 SMBus Controller

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_4000 - 0xFEC0_4FFF` (4KB window)

Planned Features: - SMBus 2.0 compliance - Master and slave modes - Clock stretching support - Packet Error Checking (PEC) - Alert response address - Configurable clock speed - APB register interface

Applications: - System management bus communication - Sensor interfaces (temperature, voltage) - EEPROM access - Battery management - Fan control

18.3.6 3.6 UART - Universal Asynchronous Receiver/Transmitter

Status: Implemented (apb4_uart_16550 + MAS; see docs/uart_16550_mas – note the shipped block has 1/2 stop bits only and no automatic flow control, and issue #60 tracks its RTL defects) **Priority:** Medium **Address:** `0xFEC0_8000 - 0xFEC0_8FFF` (4KB window)

Planned Features: - 16550-compatible register interface - Configurable baud rate generation - 5/6/7/8 data bits - Parity: none, even, odd, mark, space - Stop bits: 1, 1.5, 2 - Hardware flow control (RTS/CTS) - FIFO buffers (16-byte TX/RX) - Interrupt generation

Applications: - Debug console - Serial communication - Modem interfaces - Legacy peripheral communication

18.3.7 3.7 SPI Controller

Status: Planned **Priority:** Low **Effort:** 5-6 weeks **Address:** TBD (not in primary ILB address map)

Planned Features: - Master mode (initially; slave mode future) - Configurable clock polarity and phase (CPOL/CPHA) - Multiple chip selects - Configurable word size (8/16/32 bits) - TX/RX FIFOs - DMA support (future) - APB register interface

Applications: - Flash memory access - ADC/DAC interfaces - Display controllers - SD card communication

18.3.8 3.8 I2C Controller

Status: Planned **Priority:** Low **Effort:** 5-7 weeks **Address:** TBD (not in primary ILB address map)

Planned Features: - I2C standard (100 kHz), fast (400 kHz), fast-plus (1 MHz) modes - Multi-master arbitration - 7-bit and 10-bit addressing - Clock stretching - General call support - APB register interface

Applications: - Sensor interfaces - EEPROM access - Multi-chip communication - System configuration

18.3.9 3.9 Watchdog Timer

Status: Planned **Priority:** Low **Effort:** 2-3 weeks **Address:** TBD (not in primary ILB address map)

Planned Features: - Configurable timeout period - Countdown counter with reload - Reset generation on timeout - Lock mechanism to prevent accidental disable - Interrupt before reset (optional warning) - APB register interface

Applications: - System fault recovery - Software hang detection - Periodic system reset - Safety-critical applications

18.3.10 3.10 Power Management / ACPI Controller

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_5000 - 0xFEC0_5FFF` (4KB window)

Planned Features: - Clock gating control per block - Power domain sequencing - Reset generation and distribution - Wake event handling - Sleep/idle mode control - ACPI-compatible registers - APB register interface

Applications: - Low-power system design - Battery-powered devices - Dynamic power management - Thermal management - OS power management interface

18.3.11 3.11 IOAPIC - I/O Advanced Programmable Interrupt Controller

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_6000 - 0xFEC0_6FFF` (4KB window)

Planned Features: - I/O APIC CSR model (register-based interface) - Multiple interrupt inputs (24+) - Programmable interrupt routing - Edge and level triggered modes - Priority-based arbitration - Interrupt masking per input - APB register interface for configuration

Applications: - Advanced interrupt routing - Multi-processor interrupt distribution - Flexible interrupt mapping - Legacy IRQ redirection - PC-compatible systems

18.3.12 3.12 Interconnect ID / Version Registers

Status: Planned – NOT present in the shipped design **Priority:** Low **Effort:** 1-2 weeks **Address:** unallocated. The shipped decoder is the generated `apbx_xbar_1to10`, whose ten windows

(0x0000-0x9FFF) are fully assigned, so adding this block needs a window assignment AND a crossbar regeneration – it cannot simply occupy 0xF000 as earlier drafts of this document assumed.

Planned Features: - Vendor ID register - Device ID register - Revision ID register - Block presence/capability bits - Configuration status registers - Debug/diagnostic registers - APB register interface

Applications: - Software block discovery - Version checking - Feature detection - Debug and diagnostics - Platform identification

---

18.4 4. Integration and Wrapper Goals

18.4.1 4.1 Individual Block Integration

Each block is designed to be used standalone:

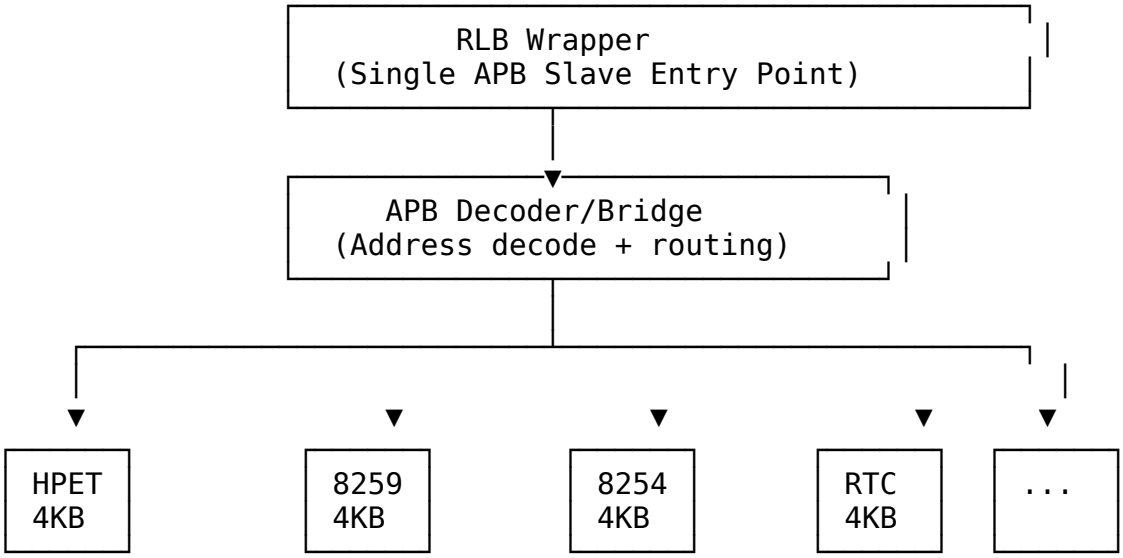
Example - HPET Integration:

```
apb4_hpet #(
    .NUM_TIMERS(3),
    .VENDOR_ID(16'h4747),
    .REVISION_ID(16'h0001),
    .CDC_ENABLE(0)
) u_hpet (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),
    // APB interface -- the ports are s_apb_P*, not bare p*
    .s_apb_PSEL     (psel_hpet),
    .s_apb_PENABLE  (penable),
    .s_apb_PREADY   (pready_hpet),
    .s_apb_PADDR    (paddr[11:0]),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PSTRB    (pstrb),
    .s_apb_PPROT    (pprot),
    .s_apb_PRDATA   (prdata_hpet),
    .s_apb_PSLVERR  (pslverr_hpet),
    // HPET-specific
    .hpet_clk       (timer_clk),
    .hpet_resetn    (timer_rst_n),
    .timer_irq    (timer_irq[2:0])
);
```

18.4.2 4.2 RLB Wrapper Architecture

Goal: Create top-level wrapper combining multiple legacy blocks into unified retro-compatible subsystem.

System Architecture:



Address Map:

Base address: `0xFEC0_0000` – the `BASE_ADDR` parameter default in `rlb_top.sv`, and the conventional location for this class of block. It is a PARAMETER: an integrator may relocate the whole window set. Window size: 4KB per block (clean power-of-2 decode)

Address Range	Block	Size	Function
`0xFEC0_0000 - 0xFEC0_0FFF`	HPET	4KB	High Precision Event Timer
`0xFEC0_1000 - 0xFEC0_1FFF`	8259 PIC (master)	4KB	Programmable Interrupt Controller
`0xFEC0_2000 - 0xFEC0_2FFF`	8254 PIT	4KB	Programmable Interval Timer
`0xFEC0_3000 - 0xFEC0_3FFF`	RTC	4KB	Real-Time Clock
`0xFEC0_4000 - 0xFEC0_4FFF`	SMBus	4KB	SMBus Host Controller
`0xFEC0_5000 - 0xFEC0_5FFF`	PM/ACPI	4KB	Power Management / ACPI

Address Range	Block	Size	Function
			Registers
0xFEC0_6000 - 0xFEC0_6FFF	IOAPIC	4KB	I/O Advanced PIC (CSR model)
0xFEC0_7000 - 0xFEC0_7FFF	GPIO	4KB	General Purpose I/O
0xFEC0_8000 - 0xFEC0_8FFF	UART 16550	4KB	Serial port
0xFEC0_9000 - 0xFEC0_9FFF	8259 PIC (slave)	4KB	Cascade slave of the PC/AT pair
All other addresses	decode-miss agent	-	Accepted and answered locally with PSLVERR

The decoder is the GENERATED `apbx_xbar_1to10` (one master, ten slaves), so the window count and the base both come from the generator invocation recorded in `rlb_top.sv`. There is no window above `0x9FFF`.

Decoder Implementation:

```
// Address decode logic (simplified)
localparam BASE_ADDR = 32'hFEC0_0000;
localparam BLOCK_SIZE = 12;  // 4KB = 2^12

logic [3:0] block_sel;
assign block_sel = paddr[15:12];  // Extract window number

always_comb begin
    psel_hpet      = (block_sel == 4'h0) & psel;  // 0xFEC0_0xxx
    psel_pic8259   = (block_sel == 4'h1) & psel;  // 0xFEC0_1xxx
    psel_pit8254   = (block_sel == 4'h2) & psel;  // 0xFEC0_2xxx
    psel_rtc       = (block_sel == 4'h3) & psel;  // 0xFEC0_3xxx
    psel_smbus     = (block_sel == 4'h4) & psel;  // 0xFEC0_4xxx
    psel_pm        = (block_sel == 4'h5) & psel;  // 0xFEC0_5xxx
    psel_ioapic    = (block_sel == 4'h6) & psel;  // 0xFEC0_6xxx
    psel_gpio      = (block_sel == 4'h7) & psel;  // 0xFEC0_7xxx
    psel_uart      = (block_sel == 4'h8) & psel;  // 0xFEC0_8xxx
    psel_pic_slave = (block_sel == 4'h9) & psel;  // 0xFEC0_9xxx
    psel_error     = !(|{psel_hpet, psel_pic8259, psel_pit8254,
                         psel_rtc, psel_smbus, psel_pm, psel_ioapic,
                         psel_gpio, psel_uart, psel_pic_slave}) &
psel;
end
```

Interface: - **Single APB slave port** at base address `0xFEC0_0000` (the `BASE_ADDR` parameter) - **Aggregated interrupt output** combining all block IRQs - **Per-block clock/reset control** for power management - **External I/O signals** (GPIO, UART, I2C/SMBus, etc.) - **Error slave** returns SLVERR for unmapped addresses

Benefits: - Simplified system integration (single APB slave) - Consistent 4KB window addressing - Clean power-of-2 address decode - Ten 4KB windows, fully assigned; expansion needs a crossbar regeneration - Single verification target - Drop-in retro-compatible peripheral subsystem

---

18.5 5. Design Standards

18.5.1 5.1 Reset Handling

MANDATORY: All blocks must use standardized reset macros from `rtl/amba/includes/reset_defs.svh`

Pattern:

```
`include "reset_defs.svh"

`ALWAYS_FF_RST(clk, rst_n,
    if (`RST_ASSERTED(rst_n)) begin
        r_state <= IDLE;
        r_counter <= '0;
    end else begin
        r_state <= w_next_state;
        r_counter <= r_counter + 1'b1;
    end
)
```

Why: - FPGA-friendly reset inference - Consistent synthesis behavior - Single-point reset polarity control - Better timing closure

18.5.2 5.2 Register Generation

Preferred: Use PeakRDL for register map generation

Process: 1. Define registers in SystemRDL (`.rdl` file) 2. Generate RTL using PeakRDL regblock 3. Create wrapper module connecting registers to core logic 4. Use edge detection for write strobes (not level)

Benefits: - Consistent register interface - Auto-generated documentation - Reduced manual RTL errors - Easy register map changes

18.5.3 5.3 Testbench Architecture

MANDATORY: Follow project testbench organization pattern

Structure:

```
dv/
├── tbclasses/{block}/          # Block-specific TB classes
│   ├── {block}_tb.py          # Main testbench
│   ├── {block}_tests_basic.py # Gate test suite
│   ├── {block}_tests_medium.py # Func test suite
│   └── {block}_tests_full.py  # Full test suite
└── tests/{block}/             # Test runners
    ├── test_apb4_{block}.py   # Pytest wrapper
    └── conftest.py            # Pytest configuration
```

Import Pattern:

```
# Always import from PROJECT AREA
from projects.components.retro_legacy_blocks.dv.tbclasses.{block}.
{block}_tb import {Block}TB
```

Test Levels: - **Gate:** Core functionality (register access, basic operation) - **Func:** Extended features (modes, configurations, edge cases) - **Full:** Stress testing, CDC variants, corner cases

Target: 100% pass rate at all levels

18.5.4 5.4 FPGA Synthesis Attributes

MANDATORY: Add FPGA synthesis hints for memory arrays

```
`ifdef XILINX
    (* ram_style = "auto" *)
`elsif SYNTH_PRAGMA
    /* synthesis ramstyle = "AUTO" */
`endif
logic [DATA_WIDTH-1:0] mem [DEPTH];
```

18.5.5 5.5 Documentation Requirements

Each block must have: - RTL comments (inline) - Register map specification - Block-level specification in `docs/{block}_mas/` - Integration guide - Test plan and results

---

18.6 6. Quality Metrics

18.6.1 6.1 Production Readiness Criteria

A block is considered “Production Ready” when:

- All gate tests pass 100%
- All func tests pass 100%

- All full tests pass ≥95%
- Complete register map specification
- RTL lint clean (Verilator)
- Reset macros used throughout
- FPGA synthesis attributes applied
- Integration guide written
- Known issues documented

18.6.2 6.2 Current Status

Block	Priority	Status	Test Pass Rate	Documentation	Production Ready
HPET	High	Implemented	see dv/tests	MAS (ch1/2/5)	RTL defects open (#46)
8259 PIC	High	Implemented	see dv/tests	MAS (ch1/5)	ISR/EOI half inert (#50)
8254 PIT	High	Implemented	6/6 both CDC configs	MAS (full)	Mode 0 only (#52)
GPIO	Medium	Implemented	see dv/tests	MAS (full)	Functional; quirks tracked (#44)
RTC	Medium	Implemented	see dv/tests	MAS (ch1/5)	Binary 24h only; time-set broken in production mode (#56)
SMBus	Medium	Implemented	see dv/tests	MAS (ch1/5)	Transfer engine non-functional (#58)
PM/ ACPI	Medium	Implemented	see dv/tests	MAS (ch1/5)	Status/W1C/wake defects (#54)
IOAPIC	Medium	Implemented	see dv/tests	MAS (ch1/2/5)	Double delivery + EOI hazards (#48)

Block	Priority	Status	Test Pass Rate	Documentation	Production Ready
UART	Medium	Implemented	see dv/tests	MAS (full)	FE/BI dead, IER unwired (#60)
SPI	Low	Planned	N/A	N/A	No
I2C	Low	Planned	N/A	N/A	No
Watchdog	Low	Planned	N/A	N/A	No
Interconnect	Low	Planned	N/A	N/A	No

18.7 7. Development Roadmap

18.7.1 7.1 Phase 1: Foundation (Complete)

- HPET implementation
- Directory structure for multiple blocks
- Testbench architecture established
- Documentation templates
- Build and test infrastructure

18.7.2 7.2 Phase 2: Core Peripherals (Next 6-9 Months)

Q1 2026 (High Priority): - 8259 PIC (6-8 weeks) - Interrupt controller - 8254 PIT (4-5 weeks) - Interval timer - RTC (3-4 weeks) - Real-time clock

Q2 2026 (Medium Priority): - GPIO Controller (4-6 weeks) - SMBus Controller (6-8 weeks) - PM/ACPI Controller (8-10 weeks)

Q3 2026: - UART (4-5 weeks) - IOAPIC (6-8 weeks)

18.7.3 7.3 Phase 3: Advanced Peripherals (9-15 Months)

Q4 2026: - SPI Controller (5-6 weeks) - I2C Controller (5-7 weeks) - Watchdog Timer (2-3 weeks)

Q1 2027: - Interconnect ID/Version Registers (1-2 weeks) - ILB Wrapper integration starts

18.7.4 7.4 Phase 4: System Integration (15+ Months)

Q2-Q4 2027: - Complete ILB wrapper with all blocks - System-level integration examples - Performance characterization - FPGA reference designs - Application notes - Software driver examples

---

18.8 8. References

18.8.1 8.1 External Standards

Peripheral Specifications: - ACPI HPET Specification 1.0a - SMBus Specification Version 2.0 - 16550 UART Datasheet - I2C Specification (NXP) - SPI Protocol Specification

Bus Protocols: - AMBA APB Protocol Specification (ARM) - AMBA 3 APB Protocol v1.0

18.8.2 8.2 Internal Documentation

- `/CLAUDE.md` - Repository AI guide
- `/PRD.md` - Master repository requirements
- `projects/components/legacy-ip/retro_legacy_blocks/CLAUDE.md` - Component AI guide
- `projects/components/legacy-ip/retro_legacy_blocks/README.md` - Component overview
- `vault/Tasks/RLB/` - task tracking (cross-block); `vault/Tasks/RLB/hpet/` for HPET

18.8.3 8.3 Block-Specific Documentation

HPET: - `docs/hpet_mas/hpet_mas_index.md` - HPET specification - `docs/IMPLEMENTATION_STATUS.md` - HPET test results - `known_issues/` - HPET issue tracking

---

18.9 9. Success Criteria

18.9.1 9.1 Individual Block Success

Each block must: - Pass all gate/func tests at 100% - Pass full tests at ≥95% - Have complete register map specification - Include integration guide with examples - Be lint-clean (Verilator) - Use reset macros throughout - Include FPGA synthesis attributes

18.9.29.2 Collection Success

The retro_legacy_blocks component is successful when: - At least 6 blocks production-ready (HPET + 5 high/medium priority blocks) - All blocks follow consistent architecture (reset macros, PeakRDL, APB interface) - RLB wrapper integrates all blocks seamlessly with clean 4KB addressing - System-level integration example provided - Complete documentation for all blocks - FPGA reference design available - Address map covers all essential retro-compatible peripherals

18.9.39.3 Long-Term Vision

Ultimate goal: - Production-quality retro-compatible peripheral subsystem - Complete peripheral coverage for legacy platform requirements - Used in production FPGA designs - Educational resource for classic peripheral design - Foundation for mixed-vintage SoC designs

---

Version: 1.0 **Last Review:** 2025-10-29 **Next Review:** After each new block completion **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

19 RLB (Retro Legacy Blocks) - FPGA Implementation Guide

Date: 2025-11-16 **Purpose:** Explain RLB blocks and FPGA implementation compatibility

---

19.1 What Are RLB (Retro Legacy Blocks)?

19.1.1 Overview

Retro Legacy Blocks are modern implementations of classic PC/embedded system peripherals, designed as reusable IP cores with APB interfaces. They provide functionality compatible with legacy systems but implemented in clean, modern SystemVerilog.

19.1.2 Purpose

Educational: - Learn how classic PC peripherals work internally - Understand interrupt controllers, timers, power management - Study register-based peripheral design - Practice verification of complex protocols

Practical: - Build soft-core SoC systems on FPGAs - Create retro computer replicas (PC-compatible) - Implement embedded systems with familiar peripherals - Rapid prototyping of control systems

Industrial: - Reusable IP cores for custom SoCs - Known-good peripheral implementations - Well-documented, tested components - Modern coding standards with legacy compatibility

---

19.2 Implemented RLB Modules

19.2.1 Complete Modules

19.2.1.1 1. HPET (High Precision Event Timer)

- **Purpose:** High-resolution timer (sub-microsecond precision)
- **Features:** Multiple timers, one-shot/periodic modes, 64-bit counters
- **PC Equivalent:** Modern replacement for PIT
- **Use Cases:** Precise timing, event scheduling, benchmarking

19.2.1.2 2. PIT_8254 (Programmable Interval Timer)

- **Purpose:** Classic PC timer (3 channels)
- **Features:** Mode 0-5 operation, binary/BCD counting
- **PC Equivalent:** 8254 PIT

- **Use Cases:** System tick, speaker control, legacy PC compatibility

19.2.1.3 3. RTC (Real-Time Clock)

- **Purpose:** Calendar and alarm functions
- **Features:** Time/date tracking, alarm, periodic interrupts
- **PC Equivalent:** MC146818 / DS12887
- **Use Cases:** Wall-clock time, wake-on-alarm, timestamping

19.2.1.4 4. PIC_8259 (Programmable Interrupt Controller)

- **Purpose:** Legacy interrupt management
- **Features:** 8 IRQ inputs, priority, cascading, masking
- **PC Equivalent:** 8259A PIC
- **Use Cases:** Legacy PC interrupts, simple systems

19.2.1.5 5. SMBus (System Management Bus)

- **Purpose:** Low-speed system management communication
- **Features:** I2C/SMBus 2.0, master/slave, PEC, FIFOs
- **PC Equivalent:** Modern motherboard SMBus controller
- **Use Cases:** Sensor monitoring, battery management, system config

19.2.1.6 6. PM_ACPI (Power Management ACPI)

- **Purpose:** Advanced power management
- **Features:** PM timer, power states (S0/S1/S3), GPE, clock gating, power domains
- **PC Equivalent:** ACPI PM controller
- **Use Cases:** Low-power systems, battery devices, sleep/wake, clock control

19.2.1.7 7. IOAPIC (I/O Advanced Programmable Interrupt Controller)

- **Purpose:** Advanced interrupt routing
- **Features:** 24 IRQs, redirection table, edge/level, priority, EOI
- **PC Equivalent:** 82093AA I/O APIC
- **Use Cases:** Multi-processor systems, flexible IRQ routing, modern PCs

---

19.3 FPGA Implementation - Nexys A7 / Genesys2

19.3.1 Can RLB Be Implemented on These Boards? YES!

All RLB modules are **100% compatible** with Nexys A7 and Genesys2 FPGAs.

19.3.2 Target FPGA Specifications

19.3.2.1 Nexys A7 (Artix-7)

- **FPGA:** XC7A100T-1CSG324C (or XC7A50T)
- **Logic Cells:** 101,440 (A100T) or 52,160 (A50T)
- **Block RAM:** 4,860 Kb
- **Clock:** 100 MHz oscillator
- **I/O:** Extensive (PMOD, switches, LEDs, UART, etc.)
- **Verdict: Excellent fit for full RLB system**

19.3.2.2 Genesys 2 (Kintex-7)

- **FPGA:** XC7K325T-2FFG900C
- **Logic Cells:** 326,080
- **Block RAM:** 16,020 Kb
- **Clock:** 200 MHz oscillator
- **I/O:** Very extensive
- **Verdict: More than sufficient, ideal for complex SoC**

19.3.3 Resource Usage Estimates

Per Module (Approximate):

Module	LUTs	FFs	BRAM	Notes
HPET	~800-1200	~600-900	0	Multi-timer, 64-bit counters
PIT_8254	~400-600	~300-500	0	3 channels, modes
RTC	~300-500	~250-400	0	Time/date logic
PIC_8259	~200-400	~150-300	0	Simple priority logic
SMBus	~600-900	~500-700	0.5	FIFOs, protocol FSM
PM_ACPI	~500-800	~400-600	0	Power FSM,

Module	LUTs	FFs	BRAM	Notes
IOAPIC	~800-1200	~600-900	0	GPE 24 IRQs, redirection

Total RLB System (~5500 LUTs, ~4300 FFs): - **Nexys A7-100T:** ~5% logic utilization Plenty of room - **Nexys A7-50T:** ~11% logic utilization Still comfortable - **Genesys 2:** ~2% logic utilization Massive headroom

19.3.4 What Can Fit on These Boards?

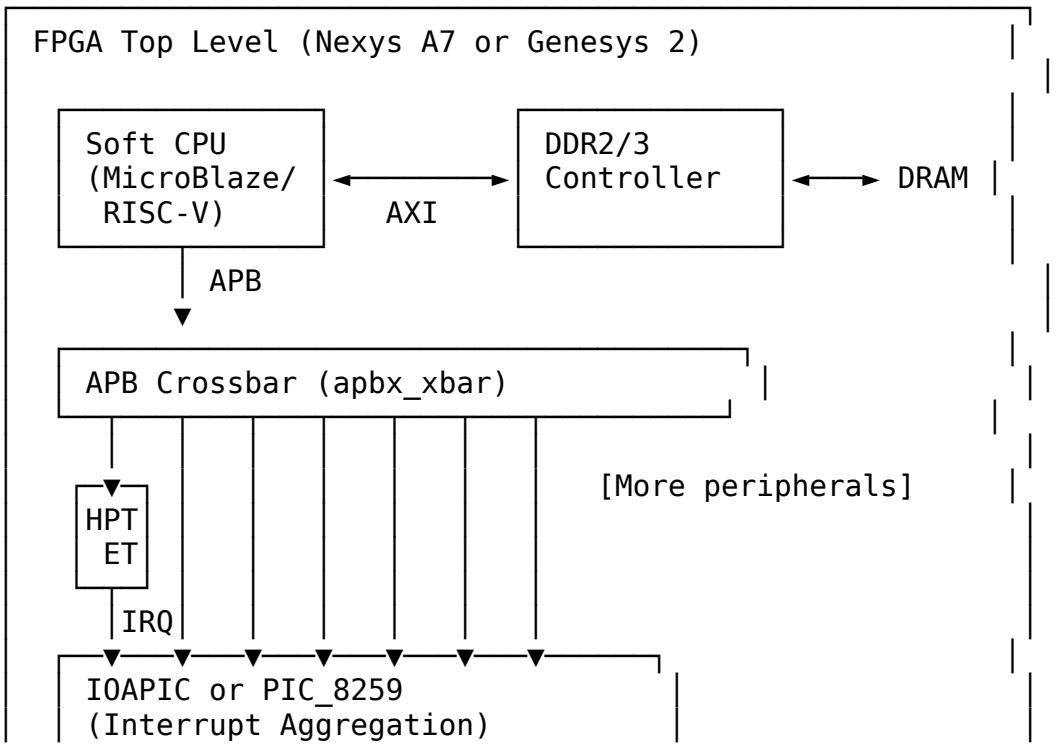
Nexys A7-100T Can Support: - All 7 RLB modules - MicroBlaze or RISC-V soft processor - DDR2 controller - Ethernet MAC - VGA/HDMI display controller - USB interface - Audio codec - **Result:** Complete retro-PC or modern embedded system

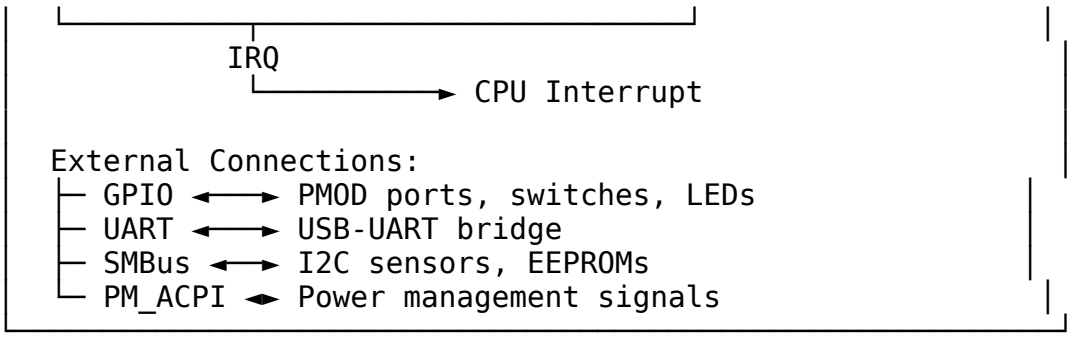
Genesys 2 Can Support: - Everything above PLUS: - Multiple soft processors - PCIe endpoint - High-speed interfaces - Complex video processing - **Result:** Professional-grade SoC development platform

---

19.4 RLB System Integration on FPGA

19.4.1 Typical FPGA System Architecture





19.4.2 Example Use Cases on FPGAs

19.4.2.1 1. Retro PC Replica

Build a PC-compatible system on FPGA: - Soft x86 core (or compatible RISC-V with x86 emulation) - All RLB peripherals for PC compatibility - VGA output for display - PS/2 keyboard/mouse - Run DOS, early Windows, Linux

19.4.2.2 2. Embedded Control System

Industrial/automotive control: - RISC-V processor - HPET for precise timing - GPIO for sensors/actuators - SMBus for temperature/voltage monitoring - PM_ACPI for power optimization - UART for debug/communication

19.4.2.3 3. Education Platform

Learn computer architecture: - Study peripheral internals - Practice interrupt handling - Understand power management - Experiment with different configurations - Debug with real hardware

19.4.2.4 4. SoC Prototyping

Rapid prototyping: - Test peripheral configurations - Validate interrupt routing - Optimize power consumption - Benchmark performance - Hardware/software co-development

---

19.5 Nexys A7 Specific Integration

19.5.1 Available Resources on Nexys A7

Peripherals Already on Board: - 16 switches (GPIO inputs) - 16 LEDs (GPIO outputs) - 5 push buttons - 7-segment displays (4 digits) - USB-UART bridge - PMOD connectors - VGA output - Microphone - Speaker - Temperature sensor - Accelerometer

19.5.2 How RLB Modules Map to Nexys A7

RLB Module	Nexys A7 Resource	Connection
GPIO	Switches, LEDs, Buttons	Direct mapping
UART_16550	USB-UART bridge	FT2232HQ chip
HPET	Internal timing	No external pins needed
PIT_8254	Internal timing	Timer/speaker
RTC	Internal clock	Optional ext crystal
PIC/IOAPIC	Internal routing	IRQ aggregation
SMBus	I2C PMOD	Via I2C PMOD adapter
PM_ACPI	Power control	Clock gates, power logic

19.5.3 Example Nexys A7 Top-Level

```
module nexys_a7_rlb_system (
    // Clock and reset
    input  wire         clk_100mhz,    // 100 MHz oscillator
    input  wire         cpu_resetn,    // Reset button

    // GPIO
    input  wire [15:0]  sw,            // Switches → GPIO inputs
    output wire [15:0]  led,           // LEDs ← GPIO outputs
    input  wire [4:0]   btn,           // Buttons → GPIO/interrupts

    // UART
    input  wire         uart_rxd,      // USB-UART RX
    output wire         uart_txd,      // USB-UART TX

    // I2C/SMBus (via PMOD)
    inout  wire         smbus_sda,
    inout  wire         smbus_scl,

    // 7-segment display
    output wire [6:0]   seg,
    output wire [7:0]   an,

    // VGA (optional)
    output wire [3:0]   vga_r, vga_g, vga_b,
```

```
    output wire         vga_hs, vga_vs
);

    // Instantiate soft processor (MicroBlaze, RISC-V, etc.)
    // Instantiate APB crossbar
    // Instantiate all RLB modules
    // Connect to board resources

endmodule
```

19.6 Implementation Recommendations

19.6.1 For Nexys A7-100T

Recommended RLB Configuration:

```
HPET        - System timing
PIT_8254    - Legacy timer for compatibility
RTC         - Real-time clock
PIC_8259    - Simple interrupt controller (OR IOAPIC)
GPIO        - Board I/O (switches, LEDs, buttons)
UART_16550  - Serial communication
SMBus       - I2C peripherals
PM_ACPI     - Optional (if power features needed)
```

Resource Budget: - RLB modules: ~6,000 LUTs, ~5,000 FFs - MicroBlaze: ~2,000-3,000 LUTs - DDR2 Controller: ~2,000 LUTs - VGA/Display: ~1,000 LUTs - **Total:** ~11,000-12,000 LUTs (11% of XC7A100T)

Plenty of room for: - User applications - Additional peripherals - Debug logic - Custom IP cores

19.6.2 For Genesys 2

Can Support Everything: - All RLB modules simultaneously - Multiple soft processors - Complex memory hierarchy - High-speed peripherals - Video processing - Network interfaces

Recommended for: - Professional SoC development - Multi-core systems - High-performance applications - Complex retro systems (full PC replica)

19.7 FPGA-Specific Considerations

19.7.1 Clock Management

Nexys A7: - 100 MHz main clock → Can generate all needed frequencies - Use MMCM/PLL for: - CPU clock (50-200 MHz) - Peripheral clocks (25-100 MHz) - Display clocks (25.175 MHz for VGA) - USB clock (60 MHz)

RLB Modules: Most work at system clock (50-100 MHz), some support CDC for different clocks

19.7.2 Power Management

PM_ACPI Use on FPGA: - Clock gating controls → Xilin x clock enables - Power domains → Can't control FPGA power, but can simulate - Useful for: Power-aware designs, testing, learning ACPI - Can control external power via GPIO pins

19.7.3 Interrupt Routing

IOAPIC vs PIC_8259 Choice: - **Simple systems (single CPU):** Use PIC_8259 (smaller) - **Advanced systems:** Use IOAPIC (more flexible, PC-compatible) - **Maximum flexibility:** Include both, use jumpers/config

19.7.4 External Interfaces

How to Connect RLB to Board Resources:

1. **GPIO Module:**
   - Inputs: Switches, buttons
   - Outputs: LEDs, 7-segment displays
   - Bidirectional: PMOD I/O pins
2. **UART_16550:**
   - Connect to FT2232HQ USB-UART
   - Baud rate generator uses system clock
   - CTS/RTS for flow control (optional)
3. **SMBus:**
   - Connect to I2C PMOD
   - On-board: Temperature sensor (ADT7420)
   - On-board: Accelerometer (ADXL362)
   - External: I2C EEPROMs, sensors via PMOD
4. **Timers (HPET/PIT):**
   - Internal only, no external pins
   - Can drive speaker output

- Can generate PWM via GPIO

5. **RTC:**
   - Internal timekeeping
   - Optional: External 32.768 kHz crystal
   - Battery backup simulation

19.8 Soft Processor Options

19.8.1 MicroBlaze (Xilinx)

- **Pros:** Well-supported, debugger, IP integration
- **Cons:** Proprietary, Vivado-specific
- **RLB Compatibility:** Excellent (APB via AXI interconnect)

19.8.2 RISC-V (Open Source)

- **Options:** VexRiscv, PicoRV32, Rocket, BOOM
- **Pros:** Open-source, portable, modern ISA
- **Cons:** Need to build toolchain
- **RLB Compatibility:** Excellent (APB native or via AXI)

19.8.3 Custom Soft Core

- Can design custom CPU specifically for RLB peripherals
- Full control over instruction set
- Educational value

19.9 Step-by-Step FPGA Implementation

19.9.1 Phase 1: Basic System (Week 1-2)

1. **Create FPGA project** (Vivado for Xilinx)
2. **Add simple processor** (MicroBlaze or RISC-V)
3. **Add APB crossbar** components/fabric-gen-ip/lattice-apbx-xbar/
4. **Add one RLB module** (start with GPIO)
5. **Connect to LEDs/switches**
6. **Test basic register access**

19.9.2 Phase 2: Expand Peripherals (Week 3-4)

7. **Add UART_16550** → Test serial communication
8. **Add HPET** → Test precise timing
9. **Add PIC_8259 or IOAPIC** → Test interrupts
10. **Add RTC** → Test time keeping

19.9.3 Phase 3: Advanced Features (Week 5-6)

11. **Add SMBus** → Test I2C communication
12. **Add PM_ACPI** → Test power management
13. **Add all remaining** RLB modules
14. **System integration testing**

19.9.4 Phase 4: Application Development (Week 7+)

15. **Develop firmware** using RLB peripherals
16. **Test real-world applications**
17. **Optimize and debug**
18. **Hardware/software co-verification**

19.10 Example Address Map for FPGA

```
Base Address | Module      | Size | Description
-------------|-------------|------|------------------------
0xFEC0_0000  | HPET        | 4KB  | High Precision Timer
0xFEC0_1000  | PIC_8259    | 4KB  | Interrupt Controller (master)
0xFEC0_2000  | PIT_8254    | 4KB  | Programmable Interval Timer
0xFEC0_3000  | RTC         | 4KB  | Real-Time Clock
0xFEC0_4000  | SMBus       | 4KB  | System Management Bus
0xFEC0_5000  | PM_ACPI     | 4KB  | Power Management
0xFEC0_6000  | IOAPIC      | 4KB  | I/O APIC
0xFEC0_7000  | GPIO        | 4KB  | General Purpose I/O
0xFEC0_8000  | UART_16550  | 4KB  | Serial Port
0xFEC0_9000  | PIC_8259    | 4KB  | Interrupt Controller (cascade
slave)
```

All accessible via APB from soft processor. The base is the `BASE_ADDR` parameter of `rlb_top` (default `0xFEC00000`); the window order is the generated `apbx_xbar_1to10`'s and is not free to rearrange.

19.11 Software Development

19.11.1 Bare-Metal Firmware

Using RLB Modules in C:

```c
#include "rlb_peripherals.h"

// Initialize system
void system_init(void) {
    // Configure GPIO
    gpio_set_direction(0xFFFF0000);  // Upper 16 = outputs

    // Initialize UART
    uart_init(115200);
    uart_puts("System starting...\n");

    // Configure HPET timer
    hpet_init();
    hpet_start_timer(0, 1000000);  // 1ms periodic

    // Setup interrupts via IOAPIC
    ioapic_configure_irq(14, 0x2E, IRQ_EDGE, 0);  // IDE
    ioapic_configure_irq(15, 0x2F, IRQ_EDGE, 0);  // IDE

    // Enable power management
    pm_acpi_init();
}
```

19.11.2 Operating System Support

Can Run: - **Bare-metal:** Direct hardware access - **RTOS:** FreeRTOS, Zephyr with RLB drivers - **Linux:** With proper device drivers for RLB - **DOS/DOS-compatible:** If using x86-compatible core - **Custom OS:** Educational OS development

---

19.12 Advantages of RLB on FPGA

19.12.1 Development Benefits

1. **Rapid Prototyping:** Change peripheral config instantly (no PCB)
2. **Debugging:** ChipScope/ILA for internal signals
3. **Flexibility:** Easy to add/remove peripherals
4. **Cost-Effective:** One board, infinite configurations

5. **Educational:** See everything, change everything

19.12.2 Technical Benefits

1. **Known-Good IP:** Well-documented, tested peripherals
2. **Standard Interfaces:** APB everywhere, easy integration
3. **Scalable:** Start simple, add complexity
4. **Portable:** Pure SystemVerilog, vendor-independent
5. **Modern Design:** Clean code, proper methodology

19.12.3 Vs. Microcontroller

Why FPGA + RLB instead of MCU?

Feature	MCU	FPGA + RLB
Peripherals	Fixed	Infinite flexibility
Interrupts	Fixed routing	Programmable (IOAPIC)
Timers	Limited count	As many as needed
Clock domains	1-2	Unlimited with CDC
Custom logic	Limited	Full flexibility
Learning value	Black box	Full visibility
Cost	Lower	Higher but more capable

19.13 Conclusion

19.13.1 Can RLB Be Used on Nexys A7/Genesys2? ABSOLUTELY!

Perfect Match Because: - Plenty of logic resources - Rich I/O for peripheral connections - Clock resources for multi-domain design - Development boards with extensive support - Educational and professional use

Recommended Starting Point: 1. Nexys A7-100T for most projects 2. Genesys 2 for complex/multi-core systems 3. Start with GPIO + UART + HPET 4. Add peripherals incrementally 5. Full RLB system fits comfortably with room to spare

The RLB modules are specifically designed to be FPGA-friendly and work excellently on these development boards!

Last Updated: 2025-11-16 **Recommendation:** Nexys A7-100T is ideal for complete RLB-based SoC development